Martin-Luther-Universität Halle-Wittenberg
Faculty of Natural Science II
Institute of Physics

Physics Modelling Assistant

Using Agentic Artificial Intelligence in Physics Modelling

Bachelor's Thesis
for the Award of the Academic Degree
Bachelor of Science
in Physik und Digitale Technologien

submitted by: Janis Felix Jacobi

Supervisor 1: Prof. Dr. Niels Schröter
Supervisor 2: Prof. Dr. Samir Lounis

submission: Halle (Saale), 22nd August 2026

Contents

Figure Index

Table Index

Glossary

Agent a singular AI system which is commonly part of an Agentic AI system 3–5, 11–13, 15, 16, 18–22, 25, 26, 28, 30–34, 44

Agentic AI an AI system with enhanced capabilities (subsubsection 2.1.3) 2, 4–6, 11–13, 34

AI Artificial Intelligence 2, 4–7, 10–13, 15, 34

API key application programming interface key; a unique and secret identifier granting access to a service 4, 15, 16, 24

context-window a size describing how many tokens a LLM can handle at once 4, 7, 12, 17, 18, 20, 21, 25, 30–32, 34

LLM Large Language Model 2, 4, 5, 7–13, 15–22, 24, 25, 28, 30–34

logit a LLM's output for every token indicating how well it is suited to be the next token. Larger values indicate a better fit. [51, p. 4] 4, 8, 9

token the input for a LLM's transformation function 4, 7–12, 15, 17–21, 25, 31, 32

1. Introduction

Artificial Intelligence can be of great assistance in scientific research [81]. As shown by Chugunova et al. [19], researchers in natural science are among the groups most familiar with AI compared to other scientists. There is a wide range of applications, stretching from finding sources in the literature to aiding with the presentation of research findings. AI can be useful for forming hypotheses and evaluating them by designing and controlling practical experiments and aiding during simulations, especially with the program implementation. [81]

In the past two years automated Agentic AI research pipelines were developed with the potential to improve the work with AI in science. The first fully autonomous approach was published by Lu et al. in 2024 [59], introducing a framework called "The AI Scientist" [59]. It identifies interesting research questions, plans and executes according experiments, interprets the results, and writes a paper stating its findings via a multistep process. A similar framework is the "InternAgent" [78], which consists of the four distinct Tasks of "Survey", "Code Review", "Assessment", and "Idea Innovation" [78]. The interactions of these tasks are managed by the "Orchestration Agent" [78], which also allows for human feedback to be introduced into the system and handles how the findings of a workflow are communicated to the human scientist. This design shows some resemblance to human research groups with individuals specialized in specific aspects of the work while a supervisor coordinates the process. In tests conducted by the "InternAgent" developers [78], their system was able to outperform the "AI-Scientist-V2" [86], a successor of "The AI Scientist" [59].

The work on the "InternAgent" [78] also introduced tracking the cost of the AI usage during a run of the system. In this metric the "InternAgent" [78] outperformed the "AI-Scientist-V2" [86] again, indicating that higher costs do not necessarily yield better results.

A less automated approach was made by Shao et al. with the introduction of "SciSciGPT" [75]. This system was developed for Science of Science research, which is why "SciSciGPT" is equipped with a data management component, a literature review component, and an analytics component. Similar to the "InterAgent", an orchestrator is used, which receives the outputs of the subtasks along with an evaluation provided by a task designated to critique the outputs of other tasks. In contrast to the "AIScientist" [59] and "InternAgent" [78], "SciSciGPT" [75] does not generate its own research ideas. Instead, it responds to an input prompt, allowing for seamless integration into the work of human scientists. [75]

As of 1st June 2026, the latest advancement in the field of automated AI system was made by Louapre, Niklaus, and Tunstall presenting the "physics-intern" [58], "a multi-agent scaffolding system that takes a physics problem and works through it autonomously" [58, section 3]. Their approach is similar to previous systems, using an orchestrator to manage a research and computation component. The results are reviewed and checked against the literature, before the critique is passed to the "Strategy Planner" [58], which has access to a literature "Background Survey" [58] and can adjust the orchestration strategy. This process is repeated until the review step deems the result correct or the maximum number of iterations is reached. The "physics-intern" [58] is able to exceed the performance of the underlying LLMs used for its Agents by over 75%, highlighting the potential of orchestration frameworks [58].

This Thesis aims to provide the foundations for an Agentic AI system capable of assisting a human physicist in crafting models for their research work. The developed system, called "Physics Modelling Assistant", can be tasked with a physics problem or modelling request and crafts an according model along with its implementation in order to answer the request. The performance of the system is measured on a state of the art benchmark to provide insight into its capabilities. In contrast to the "physics-intern" [58], the Physics Modelling Assistant mostly uses weaker LLMs, aiming to explore if improvements similar to the "physics-intern"'s [58] can be achieved. The influence of various system configurations

on the performance is investigated, with focus on the provided external information. This work aims to contribute to a better understanding of the design principles underlying Agentic AI and how such systems can be used more efficient in scientific research.

2. Theoretical Foundations

Before discussing the details of AI-based systems, several key terms are introduced.

2.1. Definitions

2.1.1. Artificial Intelligence

Artificial Intelligence (AI) describes the functionality of a system. There is no universally accepted definition for AI, so for the purpose of this Thesis, the definition from Russell and Norvig [73, p. 22-26] is used. They describe AI as being a "rational actor (rational agent), which acts in a way that achieves the best result based on the expectation of what the result should be" ([73, p. 26], translated). This definition accounts for the uncertainty regarding the goal an AI system should work towards, but also allows for a variety of systems to be considered AI. Technically, every PID-controller and data processing software - see the work of Jacobi [50] for example - satisfies this definition of AI systems. However, these systems are not what is referred to as AI throughout this Thesis, instead the term describes AI systems based on deep learning and natural language processing (see [62]) that have been developed since the 2010s.

2.1.2. Large Language Model

At its core a Language Model is a "Transformer Language Model" [88] as described by Zhao et al. It operates on small words or parts of words, which are called tokens.

When a LLM receives an input string, it divides it into tokens and feeds them to the transformation function. Based on these input tokens and the previous output tokens, the model calculates the most likely next output token. The number of tokens a model can see, and therefore consider, is limited; the limit is called the context-window of the model [54]. The final result of the LLM is its calculation for the most likely sequence of output tokens based on the given input, which means it provides the best result for the given goal, thereby fulfilling the definition of AI (see subsubsection 2.1.1). The process of developing capable LLMs is beyond the scope of this Bachelor's Thesis.

The distinction between Language Models and Large Language Models is in the number of internal parameters a model has. As stated in *A Survey of Large Language Models* [88], there is no hard cut-off for a Language Model to be considered Large. Commonly, models which are considered Large Language Models have $\geq 10 \cdot 10^9$ parameters [88].

2.1.3. Agentic AI

As described by Bandi et al., Agentic AIs are "autonomous, goal-driven systems that can operate on their own for long periods, requiring minimal human supervision" [14, 3. Results]. In order to achieve this behaviour, these systems combine abilities which are beyond LLMs. Bandi et al. highlight the "strategic planning" [14, 3. Results] capabilities, the preservation of the task's context over time [14], the usage of "external tools to extend [the system's] capabilities" [14, 3. Results] and the collaboration of individual Agents (this means AI-systems) with other Agents. [14]

2.2. Decoding the Output of LLMs

In subsubsection 2.1.2, LLM is defined based on the functionality it provides. In order to understand how the behaviour of LLMs can be modified, a more formal definition is given, based on the work of Ji et al. in *Decoding as Optimisation on the Probability Simplex: From Top-K to Top-P (Nucleus) to Best-of-K Samplers* [51].

2.2.1. LLM as a Function which Assigns Probabilities

A LLM can be described as a function f with internal parameters θ. Additionally, f takes a set of input tokens $v_{in} = (i_1, ..., i_N)$, where $N \in \mathbb{N}$ is the number of input tokens, and a set of previous output tokens $v_{out,prev} = (o_{-1}, ..., o_{-M})$, where $M \in \mathbb{N}$ is the number of previous output tokens. The function $f(\theta, v_{in}, v_{out,prev})$ assigns a probability $s(v)$ to every token $v \in \mathcal{V}$, where $\mathcal{V}$ is the vocabulary of tokens known to the LLM:

$$f(\theta, v_{in}, v_{out,prev}) = \begin{pmatrix} s(v'_1) \\ ... \\ s(v'_Z) \end{pmatrix} = s(v) \quad \text{where } Z \text{ is the size of the vocabulary } \mathcal{V} \tag{2.2.1}$$

v'_i with $i \in [1, Z] \subset \mathbb{N}$ is an element from the ordered set $\mathcal{V}'$, which allows to map the rows of the probability matrix $s(v)$ to the tokens of the vocabulary. The probability indicates how well a token is suited to be the next output token [51, p. 4]. The probabilities $s(v'_i)$ are called "logits" [51, p. 4], they are not normalized by default and larger values of a logit signal a better fit for the corresponding token to be the next output token. [51]

The LLM by itself outputs a probability distribution including every token from its vocabulary. To choose a token based on this probability distribution, Ji et al. state that the following optimization problem needs to be solved, they label it the "MASTER PROBLEM" [51, p. 5]:

$$q^* = \arg\max_{q \in \Delta(\mathcal{V})} \left[\underbrace{\langle q, s \rangle}_{\sum_{v \in \mathcal{V}} q(v) \cdot s(v)} - \lambda \Omega(q) \right] \tag{2.2.2}$$

The used quantities are:

- q^* is the probability distribution maximizing Equation 2.2.2,
- $\Delta(\mathcal{V})$ is the set of all possible probability distributions,
- q is a single probability distribution for a given vocabulary $\mathcal{V}$,
- s is the initial vector of logits the LLM produces,
- $\Omega(q)$ is a regularisation function, which can push q^* towards a certain distribution,
- λ is the strength of the regularisation function, with $\lambda \geq 0$.

There are different approaches for finding q^* and eventually getting a single token from it, the ones relevant to this Thesis are introduced in the following.

2.2.2. Greedy Decoding

This method uses no special regularisation function $\Omega(q)$, and therefore sets $\lambda = 0$. This simplifies Equation 2.2.2 to:

$$q^* = \arg\max_{q \in \Delta(\mathcal{V})} \langle q, s \rangle \tag{2.2.3}$$

The resulting q^* vector places all the probability mass on the token, which was calculated to be most likely, while all other tokens get assigned a probability of 0. Therefore, the vector q^* has the value 0 in every row except the one corresponding to the most likely token, which has probability 1. This implies the next output token is always the most likely token according to the LLM's calculations. If multiple tokens have the highest probability $s(v)$, a single token is selected among them. [51, p. 10]

2.2.3. Softmax Decoding

For Softmax Decoding, the chosen regularization function is $\Omega(q) = -\sum_{v \in \mathcal{V}} q(v) \log(q(v))$, which is the negative entropy of q. Using this $\Omega(q)$ in solving Equation 2.2.2 leads to the optimal distribution q^*, where $q_t^*(v)$ is the probability for each token: [51, p. 10-12]

$$q_t^*(v) = \frac{\exp(\frac{s(v)}{\lambda})}{\sum_{u \in \mathcal{V}} \exp(\frac{s(u)}{\lambda})} \tag{2.2.4}$$

This distribution is normalised and uses the parameter λ, which controls the spread of the probability distribution q^*. For $\lambda > 1$, the probability is spread more evenly across all tokens, which results in previously unlikely tokens being more probable than before. When $\lambda < 1$, the probability distribution is sharper, with the probability mass being more concentrated around fewer tokens which already were likely before. The next output token is selected according to the final probability distribution. For the edge case $\lambda = 0$, the regularisation function is disabled (see Equation 2.2.2), which causes the Softmax Decoding to become Greedy Decoding (see subsubsection 2.2.2). [51, p. 10-12]

The parameter λ is one way of controlling the general randomness of the output. In the context of LLMs, this parameter is often associated with the creativity of a model and called **Temperature** T. [51, p. 10-12]

2.2.4. Top-k Decoding

Top-k Decoding extends Softmax Decoding (subsubsection 2.2.3) by selecting the k ($k \in \mathbb{N} \setminus k = 0$) tokens with the highest logit values [51, p. 12]. These tokens form a subset of the vocabulary $\mathcal{V}$ called $\mathcal{V}_k$ on which Softmax Decoding is performed, while the probability for every other token $v \in \mathcal{V} \setminus \mathcal{V}_k$ is set to 0. This gives the following probability distribution [51, p. 12]:

$$q_t^*(v) = \begin{cases} \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u \in \mathcal{V}_k} e^{\frac{s(u)}{\lambda}}} & \text{for } v \in \mathcal{V}_k \\ 0 & \text{else} \end{cases} \tag{2.2.5}$$

For $k \geq |\mathcal{V}|$, Top-k Decoding is equivalent to the standard Softmax Decoding (see subsubsection 2.2.3). Setting $k = 1$ results in the same functionality as Greedy Decoding (see subsubsection 2.2.2). [51, p. 12]

From Top-k Decoding the parameter **top_k** is derived, which can be used to shape the output of LLMs [51, p. 12].

2.2.5. Top-p Decoding

Top-p Decoding is another extension to Softmax Decoding (subsubsection 2.2.3), also selecting a subset of all tokens called $\mathcal{V}_p$ from the vocabulary $\mathcal{V}$. In contrast to Top-k Decoding (subsubsection 2.2.4), Top-p Decoding defines a certain amount of probability $p \in (\ 0, 1\ ]$, on which Softmax Decoding is performed. This is achieved by selecting the token with the highest probability $s(v)$ from the set $\mathcal{V} \setminus \mathcal{V}_p$ and adding it to the set $\mathcal{V}_p$, thereby removing it from $\mathcal{V} \setminus \mathcal{V}_p$. This is repeated until the sum of the probabilities of all tokens $v \in \mathcal{V}_p$ exceeds the value of p. The probability of the tokens in $\mathcal{V} \setminus \mathcal{V}_p$ is set to 0, while Softmax Decoding (subsubsection 2.2.3) is performed on the tokens in $\mathcal{V}_p$. The resulting probability distribution is ([51])

$$q_t^*(v) = \begin{cases} \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u \in \mathcal{V}_p} e^{\frac{s(u)}{\lambda}}} & \text{for } v \in \mathcal{V}_p \\ 0 & \text{else} \end{cases}. \tag{2.2.6}$$

For $p = 1$, Top-p Decoding is equivalent to Softmax Decoding, since all tokens are included in $\mathcal{V}_p$. For $p \to 0$, only the token with the highest probability gets a non 0 probability, which makes this case equivalent to Greedy Decoding (see subsubsection 2.2.2). [51]

Top-p Decoding allows for dynamic adaptation based on how sharp or spread out the probability distribution initially returned by the LLM is. For a sharp probability distribution, only the few very likely tokens are selected to the set $\mathcal{V}_p$ and Softmax Decoding (subsubsection 2.2.3) is performed on them. Whereas, for a flatter distribution, a higher number of tokens are included into the Softmax Decoding (subsubsection 2.2.3) and therefore are possible output tokens. [51]

The derived parameter, called **top_p**, is commonly used in the field of LLMs to modify a system's output [51, p. 13].

2.3. Other Influences on an AI System's Performance

As shown by numerous benchmarks ([46], [47]), different LLMs achieve different performances. Some of the important factors for these differences are discussed in the following.

2.3.1. Predicting a LLM's Performance

One of the most noticeable differences in LLMs is the number of internal parameters they use and the amount of data used in training the models. As shown by Kaplan et al. [52, p. 5], combining the number of internal parameters N a model uses and the number of tokens D used in training a model can predict the "cross-entropy loss" [52, p. 6] L of a LLM:

$$L(N, D) = \left[\left(\frac{N_C}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_C}{D} \right]^{\alpha_D} \tag{2.3.1}$$

The used quantities are:

- $L(N, D)$ is the "cross-entropy loss" [52, p. 6]. Lower values indicate a better model.
- N_C is the critical number of internal parameters, until which Equation 2.3.1 is applicable. $N_C \approx 8.8{\cdot}10^{13}$ [52].

- N is the number of internal parameters the model uses.
- D_C is the critical number of tokens, until which Equation 2.3.1 is applicable. $D_C \approx 5.4 \cdot 10^{13}$ [52].
- α_N is the exponent for predicting $L(N)$. $\alpha_N \approx 0.076$ [52].
- α_D is the exponent for predicting $L(D)$. $\alpha_D \approx 0.095$ [52].

Other relevant influences on a LLM's performance are the number of training steps and the amount of tokens used per training step (batch size) [52], as discussed by Kaplan et al. [52]. Liu et al. [57] show the importance of how the training data are composed, highlighting the increases in predicting a model's performance when considering this factor.

2.3.2. Tool Use of LLMs

Since all LLMs are trained on a finite dataset, every model has domains in which it lacks essential knowledge, limiting its ability to perform tasks in those areas effectively. To mitigate this, LLMs are provided with additional information that enables them to answer the question at hand more reliably. In order to access external information, LLMs are equipped with tools connecting to the programmed knowledge storage. As shown by Komeili, Shuster, and Weston [53] as well as Thoppilan et al. [80], equipping a LLM with some sort of web-search tool reduces the amount of false outputs a system produces. Note that tool use is one of the features making an AI system an Agentic AI system, as discussed in subsubsection 2.1.3.

Another common class of tools are file readers, providing the ability to parse various formats like PDF, JSON, or plain text, thereby granting the LLM invoking the tool access to external information. [30]

Code execution tools allow a LLM to run generated code autonomously. This introduces the problem of untrusted code execution, since the model could generate malicious code, which accesses private information or modifies important data of the system the code is executed on. Therefore, untrusted code should only be executed inside a container separating the code from important data and systems. As stated by Marchand et al. [61], these containers, called Sandboxes, are not always fully secure; strong LLMs are able to escape them occasionally.

2.3.3. Reasoning

In the context of LLMs, reasoning is creating a chain-of-thought, which means partitioning one big task into several smaller subtasks [85]. A chain-of-thought improves the performance of LLMs on tasks which require a multi-step process to obtain the correct solution, as shown by Wei et al. [85].

2.4. Orchestrating AIs to Behave as an Agentic AI

As mentioned in subsubsection 2.1.3, combining multiple AI systems into one enables the resulting system to perform more complex tasks and achieve better results. The individual AI systems of an Agentic AI system are commonly referred to as Agents. In the following, the main approaches on how Agents can work together are presented, as they were outlined by Kulkarni and Kulkarni [55].

2.4.1. Orchestration Architectures

The Sequential Pipeline [55, p. 3] is a linear chain, where each Agent is executed one after another. Individual Agents or their tools may be enabled to rerun, as long as the pipeline is still in the corresponding step. But once an Agent is done, it can not be called again in the same run. The design process is relatively simple and the resulting systems show very deterministic behaviour, which allows errors to be located easily. [55]

The Parallel Fan-Out with Merge [55, p. 4] differs from the Sequential Pipeline (paragraph 2.4.1) by allowing certain Agents to run simultaneously. After the parallel Agents are done with their tasks, their results are merged by the next sequential Agent. Introducing an Agent before the parallelization allows for each parallel Agent to only receive the information relevant to its task. Therefore, each parallel Agent has to deal with less noise and receives less input tokens, which additionally contributes to stay within the LLM's context-window. The Agent responsible for merging resolves conflicts like contradicting information of parallel Agents. The parallel design allows for the highest throughput compared to the other architectures [55, p. 7]. [55]

The Hierarchical Supervisor-Worker [55, p. 4-5] architecture introduces a supervisor Agent. It dynamically decides which Agents (Subagents) to call and which tasks each Subagent should handle. Furthermore, the supervisor Agent reviews the Subagents' results and reruns the respective tasks, if it has low confidence in the result being correct. This forms a feedback loop, which allows the Agentic AI system to dynamically assign more difficult tasks to more powerful Agents. The hierarchical architecture produces better results than the Sequential Pipeline (paragraph 2.4.1) and the parallel design (paragraph 2.4.1), but is also more costly regarding the execution time [55, p. 7]. [55]

The Reflexive Self-Correcting Loop [55, p. 5] executes the Agents in a predefined order similar to the Sequential Pipeline (paragraph 2.4.1), but adds a verification step. The verification is performed by an Agent, which either deems the result satisfactory and declares it to be the system's output, or decides it requires improvement. Before the result is reintroduced into the system, it is critiqued by stating the problems encountered, which allows for targeted corrections on the next cycle. This architecture refines the output over time rather than simply rerunning the entire system. The Reflexive Self-Correction Loop [55, p. 5] achieves the highest accuracy but also comes at the highest cost of all architectures discussed [55, p. 7]. [55]

When interpreting the performances of the architectures discussed, it should be taken into account that Kulkarni and Kulkarni [55, p. 14-15] tested them on their extraction capabilities from financial documents. Therefore, the performances could change in different domains and for different tasks. When designing Agentic AI systems, the combination of multiple architectures is possible.

2.4.2. Communication Inside Agentic AI Systems

Communication protocols are used to define how the different components of an Agentic AI system interact. An orchestration framework may implement some or all of these protocols or similar ones. The LLMs are wrapped into a layered architecture, which handles the interactions between different LLMs and manages how tools are called. [35]

The Model Context Protocol (MCP) [32] (documentation: [79]) handles how the underlying LLM of an Agent uses tools.

The Agent Communication Protocol (ACP) [32] (documentation: [56]) allows different Agents to communicate with each other. It is based on a client-server architecture and uses "RESTful" communication [37].

The Agent-to-Agent Protocol (A2A) [32] (documentation: [76]) handles Agent interactions to enable effective cooperation. This is achieved by Agents communicating their capabilities to each other [37].

The Agent Network Protocol (ANP) [32] (documentation: [18]) manages decentralised communication between Agents. It allows for secure interactions over the internet [37].

2.4.3. Memory

In the context of Agentic AI, memory is an Agent's awareness of previous actions and contents. Short-term memory includes the context of the task the Agent is currently performing. Long-term memory allows an Agent to access information it obtained during previous runs [32]. Other kinds of memory include "semantic memory" [32], "procedural memory" [32], and "episodic memory" [32], but these are beyond the scope of this Thesis.

2.5. Evaluation of AI Systems

In order to measure the influence of an AI system's configurations, a metric for the systems performance is required. One approach is testing the AI system on a set of multiple choice questions, with each answer being clearly right or wrong [45]. The total score over the set of questions indicates the system's performance. This method is used for the "GPQA Diamond" benchmark [72] and partly for the "Massive Multi-discipline Multimodal Understanding and Reasoning (MMMU)" [87] benchmark, which are both part of the Artifical Analysis *LLM Leaderboard* [3].

Another option are open answer questions, which require a written response by the tested model. This method is used by "SciBench" [82], where the answers usually are short and need to match the expected answer exactly, since a string comparison of them is performed. The performance over all questions is accumulated and provides a final evaluation score of the tested system. [82]

In order to properly assess an AI system's capability to handle more complex tasks, human grading can be employed. This is done by Arena Intelligence [1], where users write a prompt and pick the best from two anonymous responses. Human grading is also used in "SciEx" [33], where experts evaluate the LLM's results. As acknowledged by Dinh et al. [33], human grading is not feasible for large-scale benchmarking due to its time and resource inefficiency.

2.5.1. CritPt

Presented by Zhu et al., "CritPt" [89] introduces checkpoints which allow to identify a model's progress on a question. This provides a more in-depth view of the failure points of the evaluated system. The probed system is prompted with a question and generates an answer for it; afterwards, the required output format must be adopted to enable automatic evaluation. Zhu et al. describe using the checkpoints to guide the system towards the correct answer and also mention a mechanism which assists the tested system in adopting the required output format. [89]

There are three types of questions, which require either a numerical value, a symbolic expression, or executable Python code. Composite answers are only graded as correct if all individual parts are correct. [89]

CritPt tests a system on 70 problems which are publicly available via their GitHub repository [31], the displayed scores are averages over five runs of the benchmark. The answers to the problem sets are not made public to prevent models from finding them on the internet, rendering the benchmark useless. Therefore, the model's answers to the questions are submitted to an evaluation server, returning the final result of the benchmark. Users are limited to 10 submissions per 24 hours, in order to prevent leakage of the correct answers. For the same reason, no solutions or attempts at them should be publicised. [89, p. 10]

3. Design of the Physics Modelling Assistant

In order to easily enable future work, the Physics Modelling Assistant uses open-source and free-of-charge services whenever feasible.

3.1. Orchestration Framework: CrewAI

The decision was made not to design a new framework for orchestrating AIs in order to save time on this software development heavy task and instead focus on the Physics Modelling Assistant's configurations. Derouiche, Brahmi, and Mazeni [32] discuss a number of frameworks varying in complexity and beginner friendliness. For this Thesis, the CrewAI [24] framework is used for its role focused style [32], consisting of Agents and Tasks [24] which provide a level of abstraction from the technical details of the orchestration. Every Agent can be configured individually, which includes using different LLMs for each Agent. Every Task has a description and specifications regarding the output and the output format, along with the Agent designated to it [28]. On execution of a Task, its instructions are merged with the assigned Agent's, providing the input for the underlying LLM. An Agent's behaviour is defined by its role, goal, and backstory. The role is a very short description comparable to a job title. The goal specifies the actions an Agent is supposed to perform similar to a job description. Details for the Agent's behaviour are provided via the backstory argument, including preferences or advice on how to approach the Task. CrewAI allows to access any LLM, provided the user presents a valid API key for the model. The LLM can be assigned parameters including the discussed Temperature (subsubsection 2.2.3) and top_p (subsubsection 2.2.5), the number of tokens to be used, and others. Tools can be assigned to both Agents and Tasks, with CrewAI providing ready-to-use tools, but also allowing to add custom tools [25].

The organisation of multiple CrewAI Agents and Tasks is managed by the Crew [26]. As depicted in Figure 1, it is responsible for initializing and validating the Agent's and Task's configurations before launching the system. As stated by Derouiche, Brahmi, and Mazeni [32], CrewAI's orchestration combines elements from the ACP, A2A and ANP (see subsubsection 2.4.2). The CrewAI documentation [24] and the source code available on GitHub [23] show a sequential and a hierarchical execution approach. The sequential execution (see Figure 1) runs the Tasks following the order prescribed by the source code, thereby forming a sequential Pipeline (see paragraph 2.4.1). This can be modified into a Parallel Fan-Out with Merge (see paragraph 2.4.1) by allowing certain Tasks to run at the same time, which CrewAI calls asynchronous [29]. Merging is handled by a specific Task, waiting for all parallel Tasks to be done before starting its work (see Flows [27]). The hierarchical execution style (see Figure 1) of a CrewAI Crew enables a Hierarchical Supervisor-Worker architecture (see paragraph 2.4.1) by delegating Tasks, or aspects of them, to Subtasks. Implementing a Reflexive Self-Correcting Loop (see paragraph 2.4.1) with CrewAI can be achieved by making use of Flows [27], which allow for conditional calls and the coupling of multiple Crews.

In order to achieve a deterministic workflow and allow for easy adjustments to the setup, the Sequential Pipeline architecture (paragraph 2.4.1) is used for the Physics Modelling Assistant.

3.2. LLMs Used

In order to use LLMs effectively in an automated pipeline, a provider offering a programmatic access to its LLMs is required. For this Thesis, the "Scalable AI Accelerator (SAIA)" [34] hosted by the "Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen" (GWDG) [49] is employed, offering limited

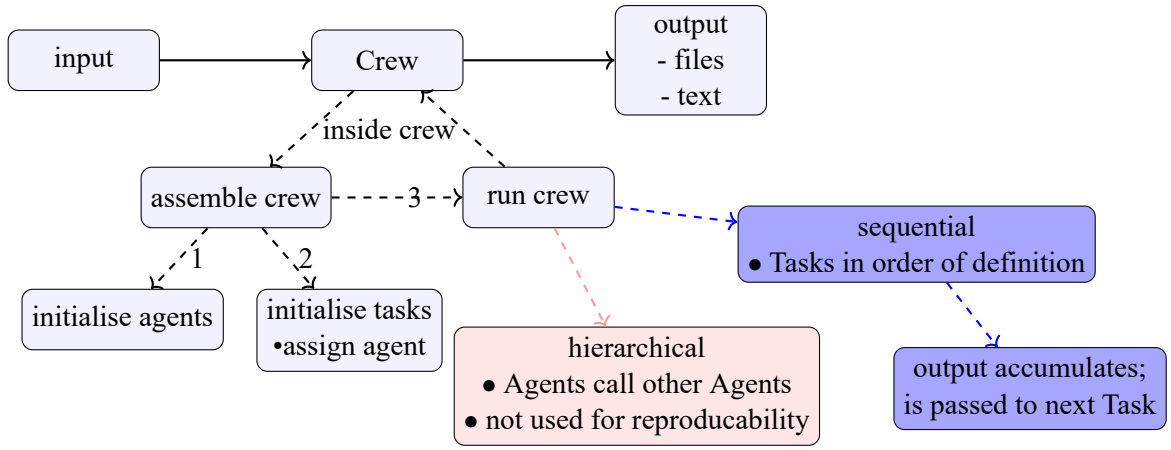

Figure 1: The main steps executed during a run of a CrewAI Crew. The Crew receives an input containing various parameters specifying its behaviour. This includes a message stating what the Crew should work on, as well as parameters shaping the LLMs' behaviour (Temperature, top_p), and the Agents' behaviour (planning, number or reiterations [20]). The Crew is assembled by initializing all of its Agents and Tasks, before mapping the Agents to their corresponding Tasks. If there is a Task without an Agent, the Crew produces an error and does not proceed with the execution. After the Crew is assembled correctly, it can run in a hierarchical or a sequential style. For the hierarchical execution one main Agent calls other (Sub-)Agents, which can call further (Sub-)Agents themselves, to aid them in accomplishing their given Task (see subsection 3.1). During the sequential execution, the Agents, respectively the Tasks with their Agents, are executed in the order they were prescribed in the source code (see subsection 3.1).

but free access to a selection of open weight LLMs. The models are accessed via an API key issued by the GWDG, the enforced rate limits are displayed in Table 1. A list of the currently available LLMs is provided on the GWDG's SAIA webpage [44], but can also be obtained via a request to the server hosting the models.

Table 1: The rate limits enforced for the chat.ai.academiccloud.de, which provides the LLMs used for the Physics Modelling Assistant. The information are obtained by making the request suggested at the GWDG's documentation page for SAIA [44, section: API Limits]. For Windows systems, the following request is suitable:
curl.exe -i ‘
» -H ”Authorization: Bearer <API_KEY>” ‘
» -H ”Content-Type: application/json” ‘
» ”https://chat-ai.academiccloud.de/v1/chat/completions”

timespan	rate limit
minute	30
hour	200
day	1000
month	3000

To identify the most capable among the available models, individual benchmarks contributing to the *LLM Leaderboard* from Artifical Analysis are considered [3]. The AA-LCR [4] tests the long context reasoning; the GPQA Diamond [6] tests for scientific reasoning capabilities; SciCode [8] evaluates the models' capabilities in science and coding; CritPt [5] (see subsubsection 2.5.1) probes the physics based reasoning abilities of the LLMs. The performances of all availabe models across the mentioned benchmarks are displayed in Table 2. During the development process of the Physics Modelling Assistant, the selection of

available LLMs changed. The DeepSeek R1 Distill Llama 70B was dropped for the higher-performing DeepSeek V4 Flash 0731. Therefore, the first benchmarking of the Physics Modelling Assistant does not use the DeepSeek V4 Flash 0731 model, but later testing explores its usefulness for the system.

Excluding the later added DeepSeek model, the benchmarking results from Table 2 indicate Qwen3.6 27B (27 billion parameters) to be the best model for long context reasoning. Qwen3.5 397B A17B (397 billion parameters in total, 17 billion active at a time) performs best in scientific reasoning tested by GPQA Diamond, while GLM 4.7 (358 billion parameters [48]) achieves the highest score on the science and coding benchmark SciCode. For the CritPt benchmark both GLM 4.7 and Qwen3.5 397B A17B achieve the best results among the tested models, with both of them scoring at only 1.7 %, indicating the difficulty of the CritPt benchmarking questions.

Therefore, Qwen3.6 27B is the LLM of choice for all Tasks dealing with large inputs, while GLM 4.7 is best suited for work which requires coding. When scientific reasoning is necessary, both Qwen3.5 397B A17B and GLM 4.7 are viable options.

Since the addition of DeepSeek V4 Flash 0731 to the GWDG service, it is the highest performing of the provided models across all benchmarks considered in Table 2, significantly outperforming all other models at CritPt with a score of 17%. It also offers the largest context-window at 1 million tokens.

Table 2: The benchmarking results for the LLMs provided by SAIA [34] are shown, the scores are extracted from Artificial Analysis [7]. The chosen benchmarks are: AA-LCR [4] evaluating long context reasoning, GPQA Diamond [6] evaluating scientific reasoning, SciCode [8] evaluating coding and science capabilities, and CritPt [5] evaluating physics reasoning. The values marked red indicate the highest performance recorded for a benchmark across the examined LLMs, excluding DeepSeek V4 Flash 0731 since it was only available after the development already began.

model	AA-LCR in %	GPQA Diamond in %	SciCode in %	CritPt in %
Apertus 70B Instruct	0	27	6	0
DeepSeek R1 Distill Llama 70B (model no longer available)	9	40	31	0
DeepSeek V4 Flash 0731 (model was added later)	74	91	50	17
Devstral 2	30	53	29	0
Gemma 4 31B	62	86	43	1
GLM 4.7	64	86	45	1.7
Llama 3.1 8B	16	26	13	0
Mistral Medium 3.5	61	75	40	0
GPT-OSS-120B (high)	51	78	39	1
Qwen3 30B A3B 2507	59	71	33	0
Qwen3 coder next	40	74	32	0
Qwen3 omni 30B A3B	0	73	31	0
Qwen3.5 122B A10B	67	86	42	1
Qwen3.5 397B A17B	66	89	42	1.7
Qwen3.6 27B	69	84	40	1
Qwen3.6 35B A3B	64	84	36	0

3.3. Tools Used

To enhance the Physics Modelling Assistant's capabilities, certain Agents are equipped with tools (see subsubsection 2.3.2). Each tool provides an object specifying the expected input and its constraints to the Agent using the tool.

3.3.1. arxiv_paper_tool with Exponential Backoff

CrewAI already provides the "arxiv_Paper_Tool" [22], enabling Agents to access the arXiv-API [10] for finding papers relevant to the question at hand. During the development process of the Physics Modelling Assistant, the CrewAI tool's requests repeatedly kept getting denied, resulting in no papers being fetched and downloaded from arXiv. As stated in the arXiv-API's Terms of Use [11], rate limits are enforced, blocking clients from sending many requests in a short amount of time. Furthermore, web servers deny repeated requests from the same client for the same service [84]. Since the CrewAI arxiv_paper_tool [22] does not implement any strategy for waiting, its requests are eventually blocked by the arXiv-API servers. As a consequence the arxiv_paper_tool was extended by implementing an exponential back-off strategy, as it is common for modern server requests [84]. Additionally to the exponential back-off, the modified tool, called arxiv_paper_tool_exponential_backoff, added a small random wait, which is common to prevent two clients requesting the same service from waiting for the same amount of time, thereby blocking each others requests repeatedly [84]. The tool finds a specified number of papers and downloads them to a designated directory.

3.3.2. pdf_reader

In order to access the papers downloaded from arXiv, a PDF reader tool was developed. It is able to read all PDF files from the specified directory, allowing the invoking Agent's LLM to see the contents of the documents. In theory, this allows the system to read sources of any number and length from the directory. In practice, however, every LLM has a context window, which limits the amount of tokens it can process. When the number of tokens read exceeds the context-window, the Agent fails its Task.

Since the tool only reads files from a hard-coded directory, it is deemed secure to be used outside of a Sandbox environment (see subsubsection 2.3.2).

3.3.3. dimensional_analysis

During the testing of the Physics Modelling Assistant, formulas from the downloaded sources were stated accurately, but assigning correct units to the individual quantities proved to be difficult for the responsible Agent. Mistakes in the unit assumptions lead to unphysical results or impossible units for the outcomes of the numerical implementation. To verify the unit consistency for all formulas used, the dimensional_analysis tool was created. It is based on the SymPy library [77] and allows an Agent to check whether the units it proposes are consistent for the relevant formulas. The tool receives an equation written as a string, a dictionary containing the used quantities and their proposed units, and a list of all units that are used. Inside the tool the units are substituted in for their respective quantities, before both sides of the formula are simplified separately. Numerical values are not considered, since the aim is only to achieve unit consistency. Lastly, the left side of the formula is divided by the right side, resulting in the unit correction required on the right side for the formula to achieve consistency. If this value is 1 then no

correction is needed, otherwise the Agent invoking the tool may apply corrections to the assumed units of the used quantities to correct the mismatch. Another conclusion the Agent may draw is to modify the tested formula by inserting physical constants which are otherwise absorbed by differing definitions of the used quantities. This allows to work with real world units while implementing a formula, which uses a different unit system.

3.3.4. Code Execution

As discussed by Marchand et al. [61], a secure Sandbox is complex to design but necessary if LLM generated code is executed directly. As stated in subsubsection 2.3.2, the weak points of Sandboxes can be exploited by capable LLMs, which is why the Physics Modelling Assistant omits code execution.

3.4. Experimental Setup for the Pipeline

The final system is designed to mirror a simplified human research workflow consisting of finding relevant information, developing a model based on the knowledge acquired, and implementing the model to provide insight into its predictive capabilities. In order to properly answer the questions from benchmarks (see subsection 2.5), the final Task is responsible for providing the output in the benchmark's required format. The recently introduced DeepSeek V4 Flash 0731 LLM is not considered for this setup for the reasons discussed in subsection 3.2.

A simplified view of the Physics Modelling Assistant's steps is shown in Figure 2, each step is performed by one Task and its corresponding Agent.

3.4.1. Input

The Physics Modelling Assistant receives the problem description, a string called *aim*, and instructions for the required format, likewise as a string. Additional parameters may be set via an interface, but default values are provided. The output directory specifies where the Tasks store their generated files; its name consists of the question number and a unique identifier, ensuring previous outputs are not overwritten. The Temperature (see subsubsection 2.2.3) and the top_p (see subsubsection 2.2.5) parameters can be modified to shape the LLM's output, along with the max_tokens parameter limiting the number of tokens a LLM produces [44].

Each Agent can perform reasoning before executing its assigned Task. Reasoning implies that an Agent creates a plan for how to approach the Task before working on it [21] (see subsubsection 2.3.3). The maximum number of reasoning attempts can be specified; otherwise the Agent retries to create a plan until an attempt is deemed sufficient [21]. An Agent is also able to rerun itself to improve its output if it determines that further improvement is required. The maximum number of iterations is set through the max_iter parameter [21], the default is 20.

3.4.2. General Parameter Choices

Since the GWDG webpage recommends Temperature and top_p values for all three LLMs considered, these defaults are retained, as they are the solutions to the optimization problem discussed in subsubsection 2.2.1,

Table 3: The default parameter settings used for the Physics Modelling Assistant.

parameter	value	justification
temperature	1.0	recommendation from the GWDG webpage [44]
top_p	0.95	recommendation from the GWDG webpage [44]
max_tokens	50k	ensuring the context-window is not exceeded
max_iter	3	balance the improvements of reiterations with the additional computation effort
reasoning	True	improves the quality of the output for complex problems
max_reasoning_attempts	3	balance the gains of reasoning with the additional computation effort

and changes would disturb the ideal configuration. For GLM 4.7 and Qwen3.6 27B the recommended parameters are Temperature = 1.0 and top_p = 0.95; the suggested Temperature for Qwen3.5 397B is lower at 0.6. For the reasons discussed in subsubsection 3.4.4, the Qwen3.5 397B model is not used in the system, which allows Temperature and top_p to be configured globally, as seen in Table 3.

The maximum number of output tokens is set to 50k (see Table 3) to ensure that the context-window is not exceeded. Qwen3.6 27B's context-window enables it to handle a maximum of 262k tokens, while Qwen3.5 397B A17B and GLM 4.7 have a context-window of 256k and 200k tokens respectively [43]. The advantages of accommodating a large number of input tokens need to be balanced with the number of output tokens being sufficient for the LLM to state its results.

As discussed in subsubsection 2.3.3, an Agent performing reasoning is generally desirable for it tends to yield improved results on complex challenges. The superiority of feedback loops mentioned in paragraph 2.4.1 indicates a positive effect of an Agent rerunning itself. However, more reasoning attempts and reiterations increase the computation effort, with all retries counting towards the rate limit shown in Table 1. Therefore, both parameters are set to 3 (see Table 3), allowing for modest dynamic improvements while minimizing the strain on the rather tight rate limit.

With one run of the Physics Modelling Assistant using eight Agents (see Figure 2) and each of them using all possible reasoning attempts and reiterations, a total of 48 requests are sent to the LLMs, with every request counting towards the providers rate limits stated in Table 1.

The Physics Modelling Agent does not use dedicated memory (subsubsection 2.4.3) but relies on CrewAI passing the outputs of previous Tasks to later ones via a shared output object (see [23, line 1610]).

3.4.3. Obtaining Information

Source Gathering In order to obtain sufficient information for building a model, the arXiv-API [10] is accessed via the modified arxiv_paper_tool (see subsubsection 3.3.1) and the papers retrieved are downloaded. The corresponding **source gatherer** Agent is based on the Qwen3.6 27B LLM and is tasked with using its tool for finding papers related to the aim. This model is used due to its superior performance in long context reasoning (see Table 2), which may also include searching for relevant information. The **gathering task** specifies that three to six papers should be found, with more papers being downloaded bearing the risk of exceeding the context-window of the LLMs used in following steps. The Task further specifies which information should be listed for each source: arXiv ID, author, date, URL, and a short summary. This content is written into a Markdown report, allowing a human to inspect the sources

found.

This Agent does not use reasoning as its Task is deemed to not require a multi step approach. All remaining parameters are kept at the global settings listed in Table 3.

Because of the issues with the arXiv-API limits [10], this Task is packaged into its own Crew, allowing it to be executed independently of all later steps. This way, changes or reruns intended to alter the behaviour of later Tasks can be made without rerunning the arXiv request again. For the evaluation of the Physics Modelling Assistant's performance in this Thesis, source gathering is executed on every retry, to provide a final result reached in one continuous run for every CritPt challenge.

Information Extraction By knowing the directory the downloaded papers are saved to, the **information extractor** Agent can invoke its pdf_reader tool (see subsubsection 3.3.2) to obtain the contents of the files in that directory. The Qwen3.6 27B LLM is used for its superior long context reasoning performance (see Table 2), along with its context-window of 262k tokens being the largest among the considered models.

The **information extractor** is instructed to find the information required to build a model that accomplishes what the aim asks for. It is specified to present its findings with proper citation and its backstory emphasises to stay true to the provided source material. The focus on stating the used sources is chosen to allow a human to verify the information stated by the LLM. The corresponding **extraction task** specifies the output to be in Markdown format, enforcing an unambiguous and human readable output.

The parameters are kept at the default settings stated in subsubsection 3.4.2.

3.4.4. Building the Model

Modelling For building the model which is at the heart of the Physics Modelling Assistant, the LLM strongest in scientific and specifically physics reasoning should be used. Therefore, the first choice was Qwen3.5 397B A17B, which performed the best at scientific reasoning (GPQA Diamond [72]) and joined best at CritPt [89], as shown in Table 2. During the testing phase, requests to this model repeatedly failed with an answer not being provided by the server before a 30 minute timeout was reached. For that reason the GLM 4.7 LLM is used instead, for it being the joined second best model at scientific reasoning (GPQA Diamond [72]) and joined best model at CritPt [89] (see Table 2). The **modeller** Agent is prompted to provide a mathematical description for the model it builds to accomplish the aim. It is further instructed to rely on the information provided by the previous extraction Task, provide a proper explanation for the steps taken, and not to write any code. The last instruction targets the Agent's shown behaviour of occasionally writing code which already implemented the model. This is not wanted due to potential modifications required for the model to be sufficient.

The **modelling task** defines the expected output to be a complete mathematical description of the model along with textual explanations. The requested format is once again Markdown, ensuring human readability of the generated output file. The default parameter settings described in Table 3 are adopted.

Unit Check The **unit checker** Agent accounts for domain specific parameter definitions and other conventions preventing a straightforward implementation of the model and its formulas. Since this requires genuine reasoning capabilities, the GLM 4.7 LLM is chosen. The goal specifies the following steps:

1. The units of all the quantities should be determined.
2. The assumed units should be checked with the dimensional_analysis tool (see subsubsection 3.3.3) to identify potential inconsistencies of the current suggestion.
3. Based on the tool's results, necessary corrections to the proposed units or the current formulas may be applied.

The **unit checking task** specifies the units of all quantities to be displayed in the Markdown output file along with the results of the tool calls made. The parameter configuration aligns with the default (see Table 3).

Parameters Suggestion In order to obtain a meaningful simulation, physically realistic starting conditions are required. The **parameter suggester** is asked to provide them along with derivations for its suggestions, justifying the use of the GLM 4.7 LLM for its reasoning capabilities. The **parameter suggestion task** emphasizes that explanations for the parameter choices are supposed to be included in the Markdown output file. The input parameters are unchanged compared to the defaults from Table 3.

3.4.5. Implementing the Model

Physics Simulation In order to capture the suggested model accurately, the **simulation physician** Agent is run with the goal of implementing the formulas of the model precisely, without making changes to them. The Agent is based on the GLM 4.7 LLM, since it performs the best in coding and science among the considered models (see Table 2). As specified by the **physician simulation task**, the implementation is supposed to use the corrected units and formulas determined by the unit checking task (see paragraph 3.4.4). To obtain human readable and interpretable results, the Task is requested to create graphics if it deems this appropriate. The output is specified to be a Python file containing executable code, with explanations added as comments. The parameters remain the same as before (see Table 3).

Simulation Correction Since the previous physician simulation task is meant to capture the model's logic into working code, it is not asked to prioritize code quality. The **simulation correcter** is tasked to correct coding mistakes and improve the code's efficiency and style, while maintaining the implemented formulas. The corresponding **simulation correction task** defines the expected output to be working Python code. The parameter configurations and LLM are unchanged compared to the simulation physician described in paragraph 3.4.5.

3.4.6. Giving a Final Answer

Benchmarks like CritPt expect a comprehensive answer (see subsection 2.5) to their questions, which is why the **final outputer** Agent is appended to the pipeline. Its goal is to provide a complete answer that obeys the requested format, which is crucial for CritPt's automated grading process [89]. The **final output task** is asked to generate an output file in Markdown format. The GLM 4.7 model is used for its performance on the CritPt benchmark stated in Table 2 and the issues with Qwen3.5 397B A17B discussed in paragraph 3.4.4.

3.4.7. Output

The result of the Physics Modelling Assistant is an object containing a string with the final Task's result, representing the answer to the defined aim. The output files generated by each Task can be inspected by a human; the Python scripts are not directly executable since they contain triple backticks at the beginning and end. Presumably, this is done to mark the section in between as code, which is not valid Python syntax. Removing or commenting the backticks results in the scripts becoming executable, apart from possible package installations.

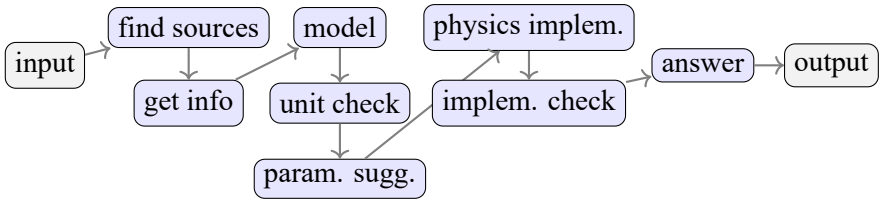

Figure 2: This figure provides an overview of all the Tasks and their order of execution, as performed by the Physics Modelling Assistant. It receives an input string which asks a question or formulates a modelling request. The system fetches papers related to the aim and downloads them from arXiv, then the papers are read and relevant information is extracted. A model is built based on the obtained information, with the model's formulas being checked for unit consistency, before starting parameters for the simulation are suggested. Only then is the model implemented into Python code, which is corrected by the following step if issues are encountered. The last Task provides the final answer to the given input ensuring it matches the required format. Each Task creates a file containing its work done during the run.

3.4.8. Connecting to CritPt's Benchmark

CritPt is based on the inspect_ai package [2], which allows to register a model by implementing the ModelAPI interface. This is achieved by the crafted PMAModelAPI class, exposing the required generate method, which receives the challenge description from CritPt, but not its number or the required output format. This does not align with the statement from subsubsection 2.5.1, suggesting the system described by Zhu et al. [89] is modified compared to the one publicly available on GitHub [31]. The same is true for CritPt guiding the system towards the correct result via checkpoints. The 70 publicly available challenges do not include any checkpoints and the GitHub repository does not mention how to obtain them [31].

The PMAModelAPI instance is programmed to identify the current challenge based on the problem description it receives, allowing it to obtain the challenge number and the required output format.

Due to conflicting dependencies of the inspect_ai and crewai packages regarding incompatible versions of the tiktoken and pydantic packages, the PMAModelAPI instance calls the Physics Modelling Assistant as a subprocess running in a different virtual environment. From the superprocess, the subprocess receives the problem description, the requested output format, and the challenge number along with a unique identifier. The latter allows for the processes to communicate via writing to and reading from a shared file, without it being overwritten by other runs of the system. This was implemented to work around the problem of extracting the Physics Modelling Assistant's output from the standard output (std_out) of the subprocess, which also contains all content that would be displayed on the terminal. Furthermore, the PMAModelAPI provides the subprocess with the values for the parameters as stated in Table 3.

The subprocess consists of two layers. The top layer handles the communication with the superprocess and triggers the bottom layer. This is the Physics Modelling Assistant which works through the steps described

in subsection 3.4 and returns its final output string to the top layer. With the superprocess waiting for the subprocess to finish, the output is written to the agreed file before completion is reported.

The superprocess obtains the output from the specified file and forwards the result to CritPt, which stores it inside a JSON-file along with some metadata. The complete set of JSON-files can be send to the evaluation server via a script provided by CritPt [31]. Since Artificial Analysis [7] has integrated CritPt into their ecosystem, a valid API key from Artificial Analysis [7] is required for the evaluation server. It is free of charge and can be obtained by creating an account on Artificial Analysis. For the evaluation requests to be approved, the account connected to the submitting API key needs to be marked as a trusted research account by Artificial Analysis [9]. For the work presented in this Thesis, presenting a valid API key enabled the necessary evaluations to be made, without explicit approval being required.

3.5. Investigating the Physics Modelling Assistant's Performance

Due to the limited access to the LLMs described in Table 1, only two full sets of answers are generated and evaluated for each configuration, instead of the five normally used for benchmarking systems with CritPt (subsubsection 2.5.1). This aims to balance the desire to test multiple configurations, with the need to obtain statistical results, and the constraints imposed by the limited access to the used LLMs.

3.5.1. An Example Run

To provide insight into the Physics Modelling Assistant's capabilities, the system is given an example modelling request. It is not taken from the CritPt benchmark, since no solutions or attempts of the same may be publicised (see subsubsection 2.5.1). The following aim is chosen:

"Calculate the Edelstein effect for a Rashba fermion (at the Gamma point of the Brillouin zone).
Compute the magnetization magnitude and direction of different directions and magnitudes of the applied electric field.
Consider how the result depends on relevant parameters of the model (e.g. chirality, Fermi velocity, spin-orbit coupling strength) and make explicit graphics."

The final output is specified to be "simple Markdown format". The parameters have the default values listed in Table 3.

This setup aims to showcase how the Physics Modelling Assistant could be used in an actual work environment, where more open end questions are explored. In contrast to that, benchmark questions (see subsection 2.5) often require a single short answer which is either correct or incorrect.

The individual Tasks in this setup are expected to behave according their configurations specified in subsubsection 3.4.3 to subsubsection 3.4.6. To enable a clearer and more measurable way of assessing each Task's behaviour, several key criteria are introduced.

Criteria for Obtaining Information For the source gathering Task, it is important that the number of papers downloaded is within the set boundaries of three to six. The information contained in the output file is evaluated based on whether it provides the required bibliographic details needed to identify each paper. All downloaded papers must be included in the report without omissions or additions, and each paper must be relevant to the specified aim.

When extracting information from the downloaded sources, citations must be provided for every stated fact. Each source must genuinely contain the information attributed to it. All content written to the output file should be relevant for constructing the model.

Criteria for Building the Model The modelling step needs to state every formula required for performing calculations with the model. It is supposed to derive additional formulas if needed and correctly cite any information that was not derived.

For every formula which is checked, the assumed unit are supposed to be stated. When verifying an expression, the Task and its Agent are evaluated on using the tool and interpreting its results correctly. In the end, all properties should have correct units with regard to the formulas they are used in.

The suggested parameters are expected to suit the described model. The individual choices need to be supported or justified.

Criteria for Implementing the Model The physics simulation task is responsible for capturing the model's logic into code. All formulas and parameters should be used according to the previous findings. The code is required to produce graphics if this is directly requested, otherwise it is optional.

After the simulation correction, all implemented formulas and there parameters are supposed to retain their previous form. Corrections should be applied to fix errors in the code and possibly improve its style, resulting in an executable script, except from the manual removal of the backticks described in subsubsection 3.4.7.

Criteria for Giving a Final Answer The output format is to be implemented correctly, providing the Physics Modelling Assistant's result for the given aim.

3.5.2. The Pipeline's Performance Compared to Individual Models

The Physics Modelling Assistant's performance in the default setup (see subsection 3.4) is measured against the CritPt benchmark (see subsubsection 2.5.1). Its results are compared to the performances of the individual LLMs used for the system.

3.5.3. The Effect of the Context-Window Length

The default setup is modified to use the DeepSeek V4 Flash 0731 model instead of the Qwen3.6 27B model for the Agent assigned to the extraction Task (see paragraph 3.4.3). Both models achieve similar scores on long context reasoning (see Table 2), but at 1 million tokens, DeepSeek V4 Flash 0731's context-window is almost four times larger than Qwen3.6 27B's. The achieved performance of this configuration is compared to the reference setup described in subsection 3.4.

3.5.4. The Performance when Using DeepSeek V4 Flash 0731

The newly added DeepSeek V4 Flash 0731 model is used for every Agent of the Physics Modelling Assistant, for the model's superior performance across all considered benchmarks (see Table 2). This configuration's performance is compared to the other setups described in subsubsection 3.5.2 and subsubsection 3.5.3.

4. Results and Discussion

4.1. Results of the Example Run from the Reference Setup

4.1.1. Results for Obtaining Information

Source Gathering A total of six papers were downloaded from arXiv (see subsubsection A.1.1). The Markdown report includes their titles, along with bibliographic information, and short summary. For the full report is available on GitHub [70].

This Task worked according to the defined criteria paragraph 3.5.1, staying within the imposed limit of downloading a maximum of six papers and creating a report that contains the required bibliographic information for just the downloaded papers. All sources directly relate to the Edelstein effect and discuss aspects of it which are relevant to the modelling request (see subsubsection 3.5.1).

Extracted Information The report on this Task includes a few additional sources. A variety of formulas are stated and explained, starting with the Rashba Hamiltonian

$$\hat{H} = \frac{p^2}{2m} + \alpha\, \hat{z} \cdot (\mathbf{p} \times \sigma), \tag{4.1.1}$$

where α is the Rashba coupling strength and σ are the Pauli-matrices. A more detailed description can be found in subsubsection A.1.2. Other information regarding energy and spin properties are stated, before analytical results for the magnetization are presented:

$$\text{HDR} \quad M_y = \frac{\mu_B |e| \tau}{2\pi}\, m\alpha\, [\hat{z} \times \mathbf{E}]_y \tag{4.1.2}$$

$$\text{LDR} \quad M_y = \frac{\mu_B |e| \tau}{2\pi}\, \sqrt{m^2\alpha^2 + 2mE_F}\, [\hat{z} \times \mathbf{E}]_y \tag{4.1.3}$$

Additionally, the Edelstein susceptibility and an anisotropic case are discussed, along with the parameter dependencies of some key formulas. Instructions and code for the implementation are provided as well. More details can be found in the appendix (subsubsection A.1.2) or on GitHub [63].

All of the additional sources stated are real scientific publications. The provided information are properly referenced to the correct papers as intended. Apart from the implementation guidelines, the provided formulas and descriptions match the intended Task output, although some of the provided information is not essential to the question at hand and its inclusion appears to be influenced by the downloaded papers discussing them. To improve the results, providing code should be discouraged, since the implemented formulas are not yet validated and therefore may be inconsistent.

4.1.2. Results for Building the Model

Modelling Since the modelling Task restated the extracted information described in paragraph 4.1.1, its output is not provided in more detail, but it can be accessed via GitHub [64]. If the file appears not to be displayed correctly on GitHub, it might be due to the used Markdown viewer. During the work on this Thesis, *Markdown Preview Enhanced* [83] was used, providing a clean human readable look except for incorrect Markdown code.

The modelling step kept the correct citations from the information extraction (see paragraph 4.1.1). The provided information being restated point to the Task and its Agent either relying too much on the information found, or deeming it sufficient for answering the modelling request. Considering the Tasks output for the benchmarking questions described in subsection 4.5, the latter appears to be the case. Contrary to the criteria stated in paragraph 3.5.1, some formulas used in following steps (see paragraph 4.1.3) were not stated or derived. This applies to the expression used for distinguishing between the high density regime (HDR) and low density regime (LDR). A possible explanation for the Task deeming the information stated by the extraction Task sufficient is the broad phrasing of the modelling request (subsubsection 3.5.1), emphasizes general dependencies to be discussed.

Unit Check The key formulas were checked for dimensional consistency, with the assumed units being stated at the start of every individual check. From the Rashba Hamiltonian, the Rashba coupling strength α was derived to have units of velocity (see subsubsection A.1.3). A factor of $\frac{1}{\hbar^2}$ was added to formula for the magnetization, resulting in

$$\text{HDR} \quad M_y = \frac{\mu_B |e| \tau}{2\pi\,\hbar^2}\, m\alpha\, [\hat{z} \times \mathbf{E}]_y \tag{4.1.4}$$

$$\text{LDR} \quad M_y = \frac{\mu_B |e| \tau}{2\pi\hbar^2}\, \sqrt{m^2\alpha^2 + 2mE_F}\, [\hat{z} \times \mathbf{E}]_y \tag{4.1.5}$$

For other formulas like the Edelstein susceptibility, no tool invocations are mentioned and no unit system was determined. More detailed information are provided in the appendix (subsubsection A.1.3) and on GitHub ([65]).

The checked formulas are essential for the model and for the Rashba Hamiltonian and the magnetization formula, the applied corrections lead to a consistent unit system (see subsubsection A.1.4). This was achieved, although the tool was not used as intended. Instead of simplifying the properties' units/dimensions to base units/dimensions, the Agent invoked the tool with more complex dimensions, like *magnetization* or *electric field*. Since the dimensional analysis tool (see subsubsection 3.3.3) does not break the provided units down, but only performs algebra on them, the tool's output did not offer the expected correction. Therefore, improvement is necessary to enable the Agent to use its tool correctly. Providing a full usage example or explicitly stating that base units are required, appear to be promising attempts.

Since no tool invocations were recorded when working on th formulas for the Edelstein susceptibility and the consistency of the initially proposed units could neither be verified nor disproven, this part of the unit check is deemed a failure.

Parameter Suggestions This Task generally acted according to its specifications, providing numeric values for each parameter relevant to the model and justifying its suggestions. However, the units determined by the previous Task were not adopted for the effective mass and Rashba coupling strength. For every suggested parameter, a justification was provided, they can be examined in subsubsection A.1.5. The full report is provided at [66].

The suggested parameters are mostly fine, except for the Rashba Coupling strength, which did not adopt the units determined by the unit check (see paragraph 4.1.2). Theses results suggest an over-reliance on using starting parameters from other sources, which is why more emphasize on adapting the found parameters to the corrected formulas from the unit check (see paragraph 4.1.2) is required. The provided justifications contain substantial mistakes, which are discussed in subsubsection A.1.5. Additionally, two of the three sources stated to support the claims state incorrect bibliographic information. To prevent the LLM from hallucinating non existent papers, enabling the corresponding Agent to use the modified

arxiv_paper_tool (see subsubsection 3.3.1) for finding real papers appears to be a viable option, since the source gathering (see paragraph 3.4.3) did not have any hallucinations when using this tool.

4.1.3. Results for Implementing the Model

Physics Simulation The simulation implemented the model using the units of from the parameter suggestion Task. A more detailed analysis is provided in subsubsection A.1.6. The full code can be inspected at [67]. An additional expression for determining whether the system is in the HDR or LDR was implemented, although it was not stated by the modelling Task (see paragraph 4.1.2). The code creates graphics for visualizing the parameter dependencies of the implemented formulas.

The generated code implements the corrected formulas for the HDR (see Equation 4.1.4) and LDR magnetization (see Equation 4.1.5). The units used for the parameters do not align with the findings of the unit check (subsubsection A.1.3). Instead, the suggestions for the starting parameters (see subsubsection A.1.5) Task were adopted. The resulting inconsistencies reflect the issues encountered with finding suitable starting parameters, rather than suggesting a fundamental flaw with adapting the model into working code. Graphics highlighting the model's parameter dependencies are generated as requested.

Simulation Correction Only insignificant alterations were made to the code generated by the previous physics simulation Task. With some manual corrections discussed in subsubsection A.1.6, Figure 3 is obtained.

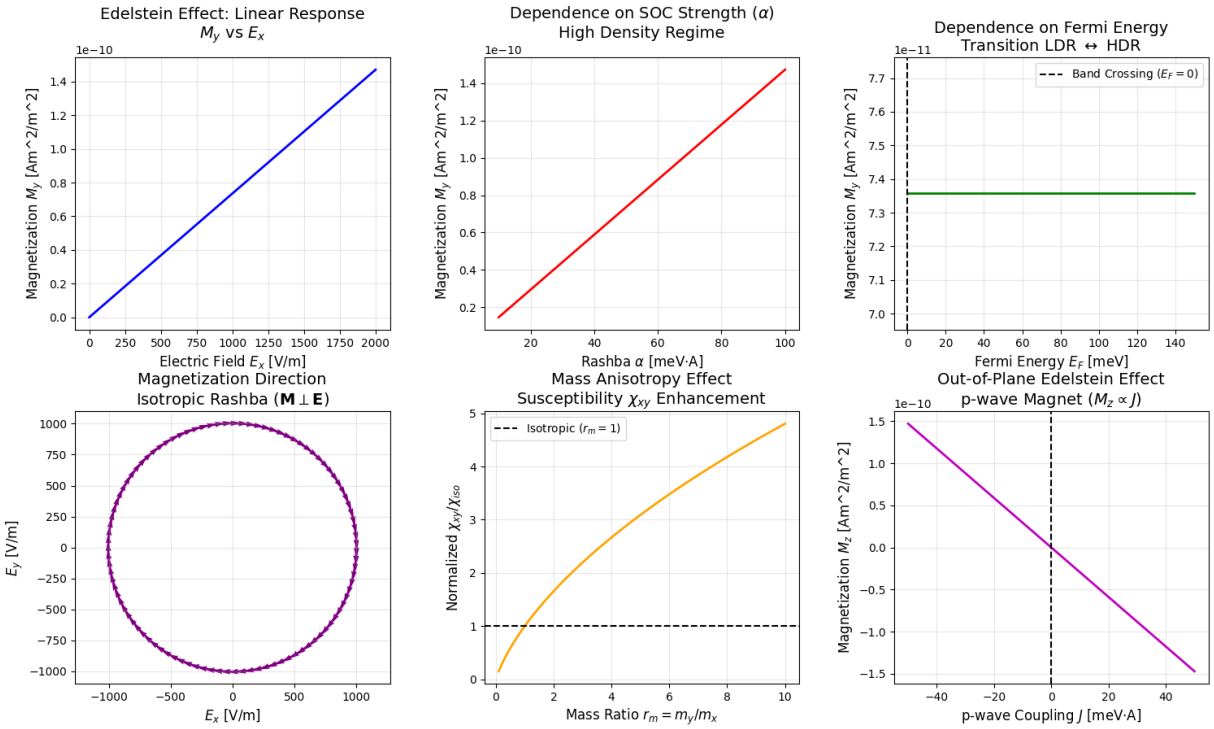

Figure 3: The graphics obtained from the Physics Modelling Assistant's simulations are shown, after manual corrections were applied.

This is acceptable considering the generated code could be executed without failures. However, some stylistic improvements regarding malformed axis labels were not implemented, neither was the issue of negative square-roots occasionally caused by the LDR magnetization formulas addressed.

4.1.4. Results for Giving a Final Answer

The final output mostly reiterated the extracted information (see paragraph 4.1.1). For the full Markdown report, see [68].

These results are in line with the expected behaviour defined in paragraph 3.5.1.

Since no specific output format nor a definitive answer to an explicit question was requested, the Task is not essential to this run of the Physics Modelling Assistant. It is valuable though for adopting the format required by the benchmarking.

4.1.5. Discussion of the Results from the Example Run

In its current configuration, the Physics Modelling Assistant properly downloads related sources from arXiv for the given input. It extracts relevant information and is able to apply unit corrections to extracted formulas, although struggling with using the tool for the unit check properly. Failures occur when finding realistic starting parameters and implementing the extracted formulas, with the identified unit corrections not being considered. The final Task providing an answer to the specified aim is only relevant if a concrete question and answer format are provided, not for a broad modelling request.

Future work should address the system not using the tool for checking units correctly, thereby not yielding meaningful results from it. The configurations for the parameters suggestion Task and Agent should emphasize the necessity of providing realistic starting parameters in the previously determined units.

The general design, yet simple, is applicable for providing relevant information and implementing basic visualizations for a given request. To assess its ability of contributing to actual research, the results from the CritPt benchmarking need to be taken into account.

4.2. Evaluation of the Physics Modelling Assistant

4.2.1. Issues Encountered during the Generation

The most common reason for failure was the API rate limit of SAIA [44] being exceeded, resulting in a 429 error code. Rarely, the context-window of a LLM was breached, resulting in failed runs. Lastly, the adoption of the output format was inconsistent. As stated in subsubsection 3.4.7, the backticks and the word “python” needed to be removed, which was accomplished via an automated script. However, some results included additional descriptions not marked as comments in front of the actual output, requiring further post generation corrections to establish the proper format. On a few occasions, the output format was not adopted at all, with only a Markdown-like descriptions being returned. If the post processing was unable to obtain output of the correct format, the challenge was retried, which also includes challenges failing due to the previously mentioned issues.

The SAIA rate limit itself can not be adjusted, as it is fixed by the GWDG. The remaining number of requests left may be checked before a run of the system is begun, to prevent attempts which can not complete from starting in the first place. Since the number of LLM calls is not fixed but depends on possible reruns of an Agent being invoked, the number of LLM usages needed can only be estimated. The possible downsides of additional LLM calls are comparably small. Considering the current maximum

amount of 48 requests being made per run (see subsubsection 3.4.2) and the rate limit of 30 calls per minute (see Table 1), a single incomplete run causes a maximum additional waiting time of two minutes. Therefore, gain of tracking the current rate usage is negligible unless the Physics Modelling Assistant is evaluated at a substantially larger scale than in this Thesis.

Failures because of the context-window being exceeded on the extraction Task (see paragraph 3.4.3) are preventable when the number of input tokens is controlled. By dividing PDF files with too many tokens into multiple chunks, single files can not exceed the context-window any more. Then the extraction Agent needs to be run multiple times, only reading an amount of tokens the underlying LLM's context-window can accommodate. To achieve this behaviour, the Physics Modelling Assistant needs to dynamically adjust how many times the information extraction Task is run.

4.3. Benchmarking Results for the Default Setup

Two independent generation runs were conducted and evaluated. As shown in Table 4, the accuracies achieved on the evaluation are 2.86 % for the first run and 0.00 % for the second. Since only two sets of answers were evaluated, the results are highly uncertain and more independent evaluation runs are required before a definitive result is obtained. The values returned by CritPt have a higher precision than the stated results, therefore, the first were used for calculating the average and sample standard deviation. The mean accuracy of (1.43 ± 2.02) % suggests that the Physics Modelling Assistant does not offer an improved performance compared to the underlying LLMs used, which are Qwen3.6 27B and GLM 4.7, scoring at 1 % and 1.7 % respectively.

The individual behaviour of the Tasks on the benchmarking questions differs from the one observed for a modelling request as described in subsection 4.1. Fewer sources are found and the information extracted is more condensed. Contrary to the behaviour described in paragraph 4.1.2, the modelling Task carries out new calculations based on the information provided, working towards solving the challenge, whereas the unit check only applies few corrections and deems most formulas consistent. Since CritPt provides explicit numerical values for the described problem [31], the Task suggesting parameters does not contribute in a meaningful way. Although not in the required format, the two coding Tasks implement the requested calculations, but also additional code visualizing the results. The answer Task is mostly able to adopt the required output format, although post generation processing is often required to remove additional content like textual descriptions or calculations not obeying the answer function required by the CritPt imposed output format.

4.4. Benchmarking Results when Using DeepSeek V4 Flash 0731 for Extracting Information

For the Crew using DeepSeek V4 Flash 0731 instead of Qwen3.6 27B for the information extraction Task (see paragraph 3.4.3), the achieved accuracies of 0.00 % and 1.43 % result in an average accuracy of (0.71 ± 1.01) % (see Table 4) when using the sample standard deviation to obtain the uncertainty. Since only two evaluations were conducted, these results are interpreted as an indicative range for the performance rather than the definitive benchmarking score for this setup. For the individual Task, a similar behaviour as discussed in subsection 4.3 was observed, supporting the claims that the stated improvements are necessary. Therefore, neither the context-window length nor the LLM used for extraction information influence the performance significantly.

The score obtained does not suggest a significant influence of using the DeepSeek V4 Flash 0731 instead

of the Qwen3.6 27B LLM on the information extraction Task. Taking the accuracy of the default setup (see subsection 4.3) into account, these findings align with the statements from the CritPt developers, suggesting that a web-search offers no substantial improvement on the CritPt challenges [89, p. 10].

It appears to be possible that the models used are lacking some of the basic knowledge required for some of the benchmarking questions and neither a regular web-search nor arXiv papers provide the required knowledge. Enabling the information extraction to access a set of physics textbooks may provide more insight, but requires the adjustments discussed in subsubsection 4.2.1 to deal with the limited context-window. Building on this idea, the system could access a retrieval argument generation (RAG) database, providing a broader selection of sources. As of 15th August 2026, among the SAIA models only E5 Mistral 7B Instruct (see [43]) explicitly performs embedding, which means turning a string into a vector which allows to search for similar ones in a vector database. Since is has a context-window of only 4096 tokens [43], it is not feasible for encoding sources into vectors. Therefore, an external RAG database needs to be accessed or a new one has to be developed.

4.4.1. Benchmarking Results when Using DeepSeek V4 Flash 0731 for Every Agent

When using DeepSeek V4 Flash 0731 as the underlying LLM for every Agent of the Physics Modelling Assistant, accuracies of 0.00 % and 0.00 % were achieved, as seen in Table 4. The resulting average accuracy and sample standard deviation of 0.00 % is not sufficient to provide a final benchmarking score and is only used as an indication for the systems capabilities. Since the uncertainty is calculated from only two measurements, a value comparable to the previously found 2 % is estimated, resulting resulting in a precision of (0.00 ± 2.02) %. With regard to the uncertainty, this configuration's performance does not differ from the other setups discussed in subsection 4.3 and subsection 4.4 significantly. Compared to the previously use Qwen3.6 27B and GLM 4.7 models both scoring less than 2 %, DeepSeek V4 Flash 0731 performs much higher on the CritPt benchmark at 20 % (see Table 2).

The performance degradation observed for CritPt when using the DeepSeek model for the Physics Modelling Assistant compared to using this model without any orchestration framework points to a negative effect of the Physics Modelling Assistant for on the LLM's capability to handle CritPt challenges. The exact failure points can not be determined from the work done in this Thesis, but an educated guess is possible. The overall design of the Physics Modelling Assistant aims at building and implementing models based on sources, not to solve a concrete question. This is reflected in the systems architecture visualized in Figure 2, using three of its eight Tasks for working towards implementing a proposed model. Finding information related to the problem before starting to tackle it is reasonable when building a scientific model. But for the CritPt challenges, a web-search does not provide a substantial boost in performance, as shown by the CriPt developers [89, p. 10] and indicated by the findings of this Thesis stated in subsection 4.4. Therefore, the Tasks responsible for finding sources and extracting information are not crucial for tackling the CritPt challenges. The modelling Task being instructed to reference its findings from the extracted information is not was needed for the CritPt benchmark either. Overall no Task of the Pipeline developed for this Thesis is suited particularly well for performing the precise calculations required. To significantly improve the system's performance, Tasks aiming to implement a model should be removed as well as the reliance of informations from the found sources. Additionally, Tasks for validating and refining the results may be added before the final output is constructed. Implementing these modifications would most certainly break the Physics Modelling Assistant's desired behaviour to provide and implement a model (see subsection 4.1) based on a broad request as seen in subsubsection 3.5.1.

If both tackling concrete challenges and crafting more general models is desired, the current Pipeline architecture is most likely insufficient due to its unadaptability to different kinds of tasks. A more flexible architecture is required, allowing the system to dynamically decide which steps are necessary for achieving

the desired result and invoking only those. The “Hierarchical Supervisor-Worker” architecture [55, p. 4-5] as described by Kulkarni and Kulkarni [55] and presented in paragraph 2.4.1, is able to provide the required adaptability. As discussed, has one main Agent deciding which Subagents are assigned which parts of the given task.

Table 4: The accuracies (acc.) for different configurations of the Physics Modelling Assistant achieved on the CritPt benchmark [89] are displayed.

configuration	1st run acc. [%]	2nd run acc. [%]	average acc. [%]
default setup (see subsubsection 3.5.2)	2.86	0.00	1.43 ± 2.02
DeepSeek V4 Flash 0731 for the extraction Agent (see subsubsection 3.5.3)	0.00	1.43	0.71 ± 1.01
DeepSeek V4 Flash 0731 for every Agent (see subsubsection 3.5.4)	0.00	0.00	0.00

4.5. Additional Remarks about Benchmarking

For the reasons discussed in subsection 3.5, this Thesis does not provide a definitive CritPt benchmarking score for the Physics Modelling Assistant, since for no configuration five independent generations and evaluations were conducted.

The times for a complete run were inconsistent, with some runs succeeding in generating output in the correct format taking less than five minutes, while other challenges took over an hour to complete. There are many factors influencing the execution time, ranging from the capabilities and usage of the system running the Physics Modelling Assistant to the time it takes the server providing the LLMs to process the request. For that reason, the times recorded throughout different runs do no provide meaningful insight into the system but rather into the environment the system lives in.

5. Conclusion and Future Work

The stated example run suggests that the Physics Modelling Assistant is generally capable of building a simple model based on arXiv [10] papers found. The model's implementation requires few manual corrections and provides a visualization. When benchmarked against CritPt [89], the system shows no significant improvements compared to the results of the individual LLMs it used internally and appears to degrade the performance of the DeepSeek V4 Flash 0731 model, pointing to the Physics Modelling Assistant's ineffectiveness for orchestrating LLMs. To verify this tendency, the system should be benchmarked with a full five runs instead of the two conducted for each setup during this Thesis.

Future improvements of the Physics Modelling Assistant should address the recorded struggles of the system to use its tool for checking the dimensional consistency of the model's formulas, as well as implementation of the model towards adopting the proposed units for the quantities used.

Future research may include a variety of tests regarding the Physics Modelling Assistant's configurations. Instead of enabling an arXiv search, a selection of physics textbooks could be provided, offering a more structured and complete knowledge foundation compared to research papers. To accommodate large inputs like textbooks, the current mechanism for reading sources needs to be modified, to prevent the context-window from being exceeded. The influence of the number of reasoning attempts and reiteration attempts the Agents are allowed to make can be investigated. As stated by Wei et al. [85], reasoning has a positive effect on a system's performance; the better performances of architectures allowing for refinement compared to those which do not [55] suggest that reiterations are beneficial as well. These tendencies could be systematically explored for the Physics Modelling Assistant to map the influences of reasoning and reiterations in more detail. To investigate the influence of configuration changes on the Physics Modelling Assistant's purpose to craft and implement models based on scientific sources, a measurement for their quality is needed to elevate such research beyond anecdotal evidence.

If the Physics Modelling Assistant is to be used effectively in a scientific work, improvements beyond modifications to the source finding and model implementation are required. The current pipeline architecture provides a very predictable behaviour but restricts the system's adaptability to different kinds of problems, which is why a more flexible design is required. To achieve this, elements of the "Hierarchical Supervisor-Worker" [55, p. 4-5] and "Reflexive Self-Correcting Loop" [55, p. 5] architectures should be implemented. This idea is supported by existing examples of Agentic AI systems like the "physics-intern" [58] and "SciSciGPT" [75]. Both of these systems dynamically decide which steps are taken towards achieving the current goal. They also implement feedback mechanisms, based on the indicated confidence in the results a rerun of one or multiple steps is conducted. Contrary to the Physics Modelling Assistant, the "physics-intern" [58] and "SciSciGPT" [75] employ less specialized Agents, enabling them to operate on broader and more complex tasks. The superiority of this architecture is backed by the CritPt benchmarking [89] results of the "physics-intern", recording improvements of up to 13.4 % in absolute and 267.5 % in relative numbers [58]. To account for possible effects regarding specific LLM and Agentic AI system combinations, testing the models used for the Physics Modelling Assistant on the "physics-intern" [58] and vice versa may be conducted.

In its current state, the Physics Modelling Assistant is more useful as a prototype rather than as a helpful assistant for a human scientist. The design process and the obtained results provide some insight into the design of Agentic AI systems, pointing to the importance of a feedback architecture which is supported by the results of Kulkarni and Kulkarni [55]. The simple design used is a reasonable starting point for systematically investigating the influences of parameters on systems combining multiple LLMs. By implementing a pipeline architecture, the current state of the Physics Modelling Assistant provides an interesting addition to the current landscape of Agentic AI frameworks for science, allowing to compare the different architectures in more detail.

References

[1] Arena Intelligence 2026, ed. *Arena*. URL: https://arena.ai/how-it-works (visited on 07/29/2026) (cit. on p. 13).

[2] AI Security Institute, UK. *Inspect AI: Framework for Large Language Model Evaluations*. 05/2024. URL: https://github.com/UKGovernmentBEIS/inspect_ai (cit. on p. 23).

[3] Artifical Analysis. *LLM Leaderboard. Comparison of AI models from OpenAI, Anthropic, Google, SpaceXAI & others*. URL: https://artificialanalysis.ai/leaderboards/models (visited on 07/12/2026) (cit. on pp. 13, 16).

[4] Artificial Analysis. *Artificial Analysis Long Context Reasoning Benchmark Leaderboard*. URL: https://artificialanalysis.ai/evaluations/artificial-analysis-long-context-reasoning (visited on 08/12/2026) (cit. on pp. 16, 17).

[5] Artificial Analysis. *CritPt Benchmark Leaderboard*. URL: https://artificialanalysis.ai/evaluations/critpt (visited on 08/12/2026) (cit. on pp. 16, 17).

[6] Artificial Analysis. *GPQA Diamond Benchmark Leaderboard*. URL: https://artificialanalysis.ai/evaluations/gpqa-diamond (visited on 08/12/2026) (cit. on pp. 16, 17).

[7] Artificial Analysis, ed. *Independent analysis of AI*. URL: https://artificialanalysis.ai/ (visited on 07/12/2026) (cit. on pp. 17, 24).

[8] Artificial Analysis. *SciCode Benchmark Leaderboard*. URL: https://artificialanalysis.ai/evaluations/scicode (visited on 08/12/2026) (cit. on pp. 16, 17).

[9] Artificial Inelligence. *CritPt Benchmark Evaluation API*. URL: https://artificialanalysis.ai/api-reference#critpt-evaluate (visited on 08/12/2026) (cit. on p. 24).

[10] arXiv. *arXiv API Access*. 07/09/2026. URL: https://info.arxiv.org/help/api/index.html (cit. on pp. 18, 20, 21, 34).

[11] arXiv. *Terms of Use for arXiv APIs*. URL: https://info.arxiv.org/help/api/tou.html (cit. on p. 18).

[12] Christian R. Ast, Jürgen Henk, Arthur Ernst, Luca Moreschini, Mihaela C. Falub, Daniela Pacilé, Patrick Bruno, Klaus Kern, and Marco Grioni. "Giant Spin Splitting through Surface Alloying". In: *Phys. Rev. Lett.* 98 (18 05/2007), p. 186807. DOI: 10.1103/PhysRevLett.98.186807. URL: https://link.aps.org/doi/10.1103/PhysRevLett.98.186807 (cit. on p. 47).

[13] Florent Auvray, Jorge Puebla, Mingran Xu, Bivas Rana, Daisuke Hashizume, and Yoshichika Otani. "Spin accumulation at nonmagnetic interface induced by direct Rashba–Edelstein effect". In: *Journal of Materials Science: Materials in Electronics* 29.18 (04/2018), pp. 15664–15670. ISSN: 1573-482X. DOI: 10.1007/s10854-018-9162-5. URL: http://dx.doi.org/10.1007/s10854-018-9162-5 (cit. on p. 42).

[14] Ajay Bandi, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala. "The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges". In: *Future Internet* 17.9 (2025). ISSN: 1999-5903. DOI: 10.3390/fi17090404. URL: https://www.mdpi.com/1999-5903/17/9/404 (cit. on p. 7).

[15] M. Ben Shalom, A. Ron, A. Palevski, and Y. Dagan. "Shubnikov–De Haas Oscillations in $SrTiO_3/LaAlO_3$ Interface". In: *Phys. Rev. Lett.* 105 (20 11/2010), p. 206401. DOI: 10.1103/PhysRevLett.105.206401. URL: https://link.aps.org/doi/10.1103/PhysRevLett.105.206401 (visited on 08/15/2026) (cit. on p. 47).

[16] M. Ben Shalom, M. Sachs, D. Rakhmilevitch, A. Palevski, and Y. Dagan. "Tuning Spin-Orbit Coupling and Superconductivity at the $SrTiO_3/LaAlO_3$ Interface: A Magnetotransport Study". In: *Phys. Rev. Lett.* 104 (12 03/2010), p. 126802. DOI: 10.1103/PhysRevLett.104.126802. URL: https://link.aps.org/doi/10.1103/PhysRevLett.104.126802 (visited on 08/21/2026) (cit. on p. 47).

[17] Yu. A. Bychkov and E. I. Rashba. "Properties of a 2D electron gas with lifted spectral degeneracy". In: *Письма в ЖЭТФ* 39 (2 1984), p. 66. URL: http://jetpletters.ru/ps/0/article_19121.shtml (cit. on p. 42).

[18] Gaowei Chang. *Agent Network Protocol*. URL: https://agent-network-protocol.com/ (visited on 07/29/2026) (cit. on p. 13).

[19] Marina Chugunova, Dietmar Harhoff, Katharina Hölzle, Verena Kaschub, Sonal Malagimani, Ulrike Morgalla, and Robert Rose. "Who uses AI in research, and for what? Large-scale survey evidence from Germany". In: *Research Policy* 55.2 (2026), p. 105381. ISSN: 0048-7333. DOI: https://doi.org/10.1016/j.respol.2025.105381. URL: https://www.sciencedirect.com/science/article/pii/S0048733325002100 (cit. on p. 5).

[20] CrewAI. *Agent/core*. URL: https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/agent/core.py (visited on 07/30/2026) (cit. on p. 16).

[21] CrewAI. *Agents*. URL: https://docs.crewai.com/v1.15.11/en/concepts/agents (visited on 08/05/2026) (cit. on p. 19).

[22] CrewAI. *ArxivPaperTool*. URL: https://github.com/crewAIInc/crewAI/tree/main/lib/crewai-tools/src/crewai_tools/tools/arxiv_paper_tool (visited on 07/30/2026) (cit. on p. 18).

[23] CrewAI. *crew.py*. 04/22/2026. URL: https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/crew.py (visited on 04/22/2026) (cit. on pp. 15, 20).

[24] CrewAI. *crewAi*. 04/22/2026. URL: https://crewai.com/ (visited on 04/22/2026) (cit. on p. 15).

[25] CrewAI. *crewai tools*. URL: https://github.com/crewAIInc/crewAI/tree/main/lib/crewai-tools/src/crewai_tools (visited on 07/30/2026) (cit. on p. 15).

[26] CrewAI. *Crews*. URL: https://docs.crewai.com/v1.15.16/en/concepts/crews (visited on 07/29/2026) (cit. on p. 15).

[27] CrewAI. *Flows*. URL: https://docs.crewai.com/v1.15.9/en/concepts/flows (visited on 07/30/2026) (cit. on p. 15).

[28] CrewAI. *task.py*. 04/22/2026. URL: https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/task.py (visited on 04/22/2026) (cit. on p. 15).

[29] CrewAI. *Tasks*. URL: https://docs.crewai.com/v1.15.9/en/concepts/tasks#asynchronous-execution (visited on 07/30/2026) (cit. on p. 15).

[30] CrewAI. *Tools*. URL: https://docs.crewai.com/v1.15.16/en/concepts/tools (visited on 08/01/2026) (cit. on p. 11).

[31] CritPt. *challanges*. 04/22/2026. URL: https://github.com/CritPt-Benchmark/CritPt/tree/main/data/public_test_challenges (visited on 04/22/2026) (cit. on pp. 14, 23, 24, 31).

[32] Hana Derouiche, Zaki Brahmi, and Haithem Mazeni. *Agentic AI Frameworks: Architectures, Protocols, and Design Challenges*. 2025. arXiv: 2508.10146 [cs.AI]. URL: https://arxiv.org/abs/2508.10146 (visited on 07/28/2026) (cit. on pp. 13, 15).

[33] Tu Anh Dinh, Carlos Mullov, Leonard Bärmann, et al. *SciEx: Benchmarking Large Language Models on Scientific Exams with Human Expert Grading and Automatic Grading*. 2024. arXiv: 2406.10421 [cs.CL]. URL: https://arxiv.org/abs/2406.10421 (cit. on p. 13).

[34] Ali Doosthosseini, Jonathan Decker, Hendrik Nolte, and Julian Kunkel. "SAIA: a seamless Slurm-native solution for HPC-based services". In: *The Journal of Supercomputing* 82.7 (05/2026), p. 403. ISSN: 1573-0484. DOI: 10.1007/s11227-026-08508-3. URL: https://doi.org/10.1007/s11227-026-08508-3 (cit. on pp. 15, 17).

[35] Yogesh K. Dwivedi, Mohamed Y. I. Helal, Ibrahim A. Elgendy, Rasha Alahmad, Paul Walton, Ayoung Suh, Vinay Singh, and Il Jeon. "Agentic AI Systems: What It Is and Isn't". In: *Global Business and Organizational Excellence* 45.3 (2026), pp. 253–263. DOI: https://doi.org/10.1002/joe.70018. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/joe.70018. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/joe.70018 (visited on 07/28/2026) (cit. on p. 12).

[36] Victor M. Edelstein. "Spin polarization of conduction electrons induced by electric current in two-dimensional asymmetric electron systems". In: *Solid State Communications* 73 (1990), pp. 233–235. URL: https://api.semanticscholar.org/CorpusID:121468791 (cit. on p. 42).

[37] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. *A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)*. 2025. arXiv: 2505.02279 [cs.AI]. URL: https://arxiv.org/abs/2505.02279 (visited on 07/29/2026) (cit. on p. 13).

[38] Hans-Andreas Engel, Emmanuel I. Rashba, and Bertrand I. Halperin. "Out-of-Plane Spin Polarization from In-Plane Electric and Magnetic Fields". In: *Physical Review Letters* 98.3 (01/2007). ISSN: 1079-7114. DOI: 10.1103/physrevlett.98.036602. URL: http://dx.doi.org/10.1103/PhysRevLett.98.036602 (cit. on p. 42).

[39] Motohiko Ezawa. "Out-of-plane Edelstein effects: Electric field induced magnetization in wave magnets". In: *Physical Review B* 111.16 (04/2025). ISSN: 2469-9969. DOI: 10.1103/physrevb.111.l161301. URL: http://dx.doi.org/10.1103/PhysRevB.111.L161301 (cit. on p. 42).

[40] Takumi Funato and Mamoru Matsuo. "Acoustic Rashba–Edelstein effect". In: *Journal of Magnetism and Magnetic Materials* 540 (12/2021), p. 168436. ISSN: 0304-8853. DOI: 10.1016/j.jmmm.2021.168436. URL: http://dx.doi.org/10.1016/j.jmmm.2021.168436 (cit. on p. 42).

[41] Irene Gaiardoni, Mattia Trama, Alfonso Maiellaro, Claudio Guarcello, Francesco Romeo, and Roberta Citro. "Edelstein Effect in Isotropic and Anisotropic Rashba Models". In: *Condensed Matter* 10.1 (03/2025), p. 15. ISSN: 2410-3896. DOI: 10.3390/condmat10010015. URL: http://dx.doi.org/10.3390/condmat10010015 (cit. on pp. 42, 45, 46).

[42] Pietro Gambardella and Ioan Miron. "Current-induced spin-orbit torques". In: *Philosophical transactions. Series A, Mathematical, physical, and engineering sciences* (08/13/2011). DOI: 10.1098/rsta.2010.0336 (cit. on p. 42).

[43] Gesellschaft für wissenschafliche Datenverarbeitung mbH Göttingen, ed. URL: https://docs.hpc.gwdg.de/services/ai-services/chat-ai/models/index.html (visited on 08/11/2026) (cit. on pp. 20, 32).

[44] Gesellschaft für wissenschafliche Datenverarbeitung mbH Göttingen, ed. *SAIA*. URL: https://docs.hpc.gwdg.de/services/ai-services/saia/index.html (visited on 07/30/2026) (cit. on pp. 16, 19, 20, 30).

[45] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. *Measuring Massive Multitask Language Understanding*. 2021. arXiv: 2009.03300 [cs.CY]. URL: https://arxiv.org/abs/2009.03300 (visited on 07/29/2026) (cit. on p. 13).

[46] HuggingFace. *Agent Arena*. URL: https://huggingface.co/spaces/lmarena-ai/arena-leaderboard (visited on 07/28/2026) (cit. on p. 10).

[47] HuggingFace. *Artificial Analysis LLM Leaderboard*. URL: https://huggingface.co/spaces/ArtificialAnalysis/LLM-Performance-Leaderboard (visited on 07/28/2026) (cit. on p. 10).

[48] HuggingFace. *GLM-4.7-FP8*. URL: https://huggingface.co/zai-org/GLM-4.7-FP8 (visited on 07/30/2026) (cit. on p. 17).

[49] *Imprint*. URL: https://gwdg.de/imprint/ (visited on 07/30/2026) (cit. on p. 15).

[50] Jens Jacobi. "Ein Pascal-orientiertes Unterstützungssystem für grafische Darstellungen und Verarbeitung von Meßdaten in der Biotechnologie". Diplomarbeit. Ingeneurshochschule Köthen
Sektion Biotechnologie, Lebensmitteltechnologie
Sektion Mathematik, Informatik, Rechentechnik, 01/1990 (cit. on p. 7).

[51] Xiaotong Ji, Rasul Tutunov, Matthieu Zimmer, and Haitham Bou-Ammar. *Decoding as Optimisation on the Probability Simplex: From Top-K to Top-P (Nucleus) to Best-of-K Samplers*. 2026. arXiv: 2602.18292 [cs.LG]. URL: https://arxiv.org/abs/2602.18292 (visited on 07/27/2026) (cit. on pp. 4, 8–10).

[52] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361 [cs.LG]. URL: https://arxiv.org/abs/2001.08361 (visited on 07/28/2026) (cit. on pp. 10, 11).

[53] Mojtaba Komeili, Kurt Shuster, and Jason Weston. *Internet-Augmented Dialogue Generation*. Ed. by Smaranda Muresan, Preslav Nakov, and Aline Villavicencio. Dublin, Ireland, 05/2022. DOI: 10.18653/v1/2022.acl-long.579. URL: https://aclanthology.org/2022.acl-long.579/ (visited on 07/28/2026) (cit. on p. 11).

[54] Vikram Krishnamurthy. *LLMs as High-Dimensional Nonlinear Autoregressive Models with Attention: Training, Alignment and Inference*. 2026. arXiv: 2602.00426 [cs.LG]. URL: https://arxiv.org/abs/2602.00426 (cit. on p. 7).

[55] Siddhant Kulkarni and Yukta Kulkarni. *Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and Production Scaling Strategies*. 2026. arXiv: 2603.22651 [cs.AI]. URL: https://arxiv.org/abs/2603.22651 (visited on 07/28/2026) (cit. on pp. 11, 12, 33, 34).

[56] Linux Foundation AI & Data. *Agent Communication Protocol*. URL: https://agentcommunicationprotocol.dev/about/mission-and-team (visited on 07/29/2026) (cit. on p. 13).

[57] Emmy Liu, Amanda Bertsch, Lintang Sutawika, et al. *Not-Just-Scaling Laws: Towards a Better Understanding of the Downstream Impact of Language Model Design Decisions*. 2026. arXiv: 2503.03862 [cs.CL]. URL: https://arxiv.org/abs/2503.03862 (visited on 07/28/2026) (cit. on p. 11).

[58] David Louapre, Joel Niklaus, and Lewis Tunstall. "physics-intern: an autonomous agentic framework for physics research". In: *Hugging Face* (04/12/2026). URL: https://huggingface.co/spaces/huggingface/physics-intern (visited on 07/24/2026) (cit. on pp. 5, 34).

[59] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: https://arxiv.org/abs/2408.06292 (cit. on p. 5).

[60] A. H. MacDonald. "Theory of High-Energy Features in the Tunneling Spectra of Quantum-Hall Systems". In: *Phys. Rev. Lett.* 105 (20 11/2010), p. 206801. DOI: 10.1103/PhysRevLett.105.206801. URL: https://link.aps.org/doi/10.1103/PhysRevLett.105.206801 (visited on 08/21/2026) (cit. on p. 47).

[61] Rahul Marchand, Art O Cathain, Jerome Wynne, Philippos Maximos Giavridis, Sam Deverett, John Wilkinson, Jason Gwartz, and Harry Coppock. *Quantifying Frontier LLM Capabilities for Container Sandbox Escape*. 2026. arXiv: 2603.02277 [cs.CR]. URL: https://arxiv.org/abs/2603.02277 (visited on 07/30/2026) (cit. on pp. 11, 19).

[62] Rahul Mundlamuri, Ganesh Reddy Gunnam, Nikhil Kumar Mysari, and Jayakanth Pujuri. "The Evolution of AI: From Classical Machine Learning to Modern Large Language Models". In: *IEEE Access* 13 (2025), pp. 178302–178341. DOI: 10.1109/ACCESS.2025.3621344 (cit. on p. 7).

[63] Physics Modelling Assistant. Ed. by Janis Jacobi. URL: https://github.com/publicFile-11/publicFiles/blob/main/version_1/information.md (cit. on pp. 27, 43).

[64] Physics Modelling Assistant. Ed. by Janis Jacobi. URL: https://github.com/publicFile-11/publicFiles/blob/main/version_1/model.md (cit. on p. 27).

[65] Physics Modelling Assistant. Ed. by Janis Jacobi. URL: https://github.com/publicFile-11/publicFiles/blob/main/version_1/unit_check.md (cit. on pp. 28, 45).

[66] Physics Modelling Assistant. Ed. by Janis Jacobi. URL: https://github.com/publicFile-11/publicFiles/blob/main/version_1/parameter_suggestion.md (cit. on pp. 28, 47).

[67] Physics Modelling Assistant. Ed. by Janis Jacobi. URL: https://github.com/publicFile-11/publicFiles/blob/main/version_1/simulation_physician.py (cit. on pp. 29, 47).

[68] Physics Modelling Assistant. Ed. by Janis Jacobi. URL: https://github.com/publicFile-11/publicFiles/blob/main/version_1/final_output.md (cit. on p. 30).

[69] Physics Modelling Assistant. Ed. by Janis Jacobi. URL: https://github.com/publicFile-11/publicFiles/blob/main/version_1/simulation_correction.py (cit. on p. 48).

[70] Physics-Modelling-Assistant. Ed. by Janis Jacobi. URL: https://github.com/publicFile-11/publicFiles/blob/main/sources.md (cit. on pp. 27, 42).

[71] E. I. Rashba and V. I. Sheka. "Electric-Dipole Spin Resonances". In: (2018). arXiv: 1812.01721 [cond-mat.mes-hall]. URL: https://arxiv.org/abs/1812.01721 (cit. on p. 42).

[72] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. *GPQA: A Graduate-Level Google-Proof Q&A Benchmark*. 2023. arXiv: 2311.12022 [cs.AI]. URL: https://arxiv.org/abs/2311.12022 (visited on 07/29/2026) (cit. on pp. 13, 21).

[73] Stuart Russell and Peter Norvig. *Künstliche Intelligenz Ein moderner Ansatz*. Pearson Benelux B.V. (Zweigniederlassung Deutschland), 2023. ISBN: 9783868944303. URL: https://elibrary.pearson.de/book/99.150005/9783863263263 (visited on 07/26/2026) (cit. on p. 7).

[74] Georg Schmidt and Niels Schröter. *Experimentalphysik C für PDT WiSe 2025/2026*. URL: https://mos.uni-halle.de/Download/Aktuell/GB/MH_13108_aktuell.pdf (visited on 08/04/2026) (cit. on p. 47).

[75] Erzhuo Shao, Yifang Wang, Yifan Qian, Zhenyu Pan, Han Liu, and Dashun Wang. "SciSciGPT: advancing human–AI collaboration in the science of science". In: *Nature Computational Science* (03/01/2026). URL: https://doi.org/10.1038/s43588-025-00906-6 (visited on 07/24/2026) (cit. on pp. 5, 34).

[76] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. *Announcing the Agent2Agent Protocol (A2A)*. 04/09/2025. URL: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/ (visited on 07/29/2026) (cit. on p. 13).

[77] SymPy Development Team. *Welcome to SymPy's documentation!* 04/27/2025. URL: https://docs.sympy.org/latest/index.html (visited on 07/09/2026) (cit. on p. 18).

[78] InternAgent Team, Bo Zhang, Shiyang Feng, et al. *InternAgent: When Agent Becomes the Scientist – Building Closed-Loop System from Hypothesis to Verification*. 2025. arXiv: 2505.16938 [cs.AI]. URL: https://arxiv.org/abs/2505.16938 (cit. on p. 5).

[79] LCC) The Linux Foundation Project (LF Projects. *Model Context Protocol*. URL: https://github.com/modelcontextprotocol (visited on 07/29/2026) (cit. on p. 13).

[80] Romal Thoppilan, Daniel De Freitas, Jamie Hall, et al. *LaMDA: Language Models for Dialog Applications*. 2022. arXiv: 2201.08239 [cs.CL]. URL: https://arxiv.org/abs/2201.08239 (visited on 07/28/2026) (cit. on p. 11).

[81] Hanchen Wang, Tianfan Fu, Yuanqi Du, et al. "Scientific discovery in the age of artificial intelligence". In: *Nature* (08/01/2023). URL: https://doi.org/10.1038/s41586-023-06221-2 (visited on 07/24/2026) (cit. on p. 5).

[82] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. *SciBench: Evaluating College-Level Scientific Problem-Solving Abilities of Large Language Models*. 2024. arXiv: 2307.10635 [cs.CL]. URL: https://arxiv.org/abs/2307.10635 (visited on 07/29/2026) (cit. on p. 13).

[83] Yiyi Wang. *Markdown Preview Enhanced*. URL: https://github.com/shd101wyy/vscode-markdown-preview-enhanced (visited on 07/17/2026) (cit. on p. 27).

[84] Sandro Wefel. *Rechnernetze und verteilte Systeme WiSe 2025/2026*. URL: https://mos.uni-halle.de/Download/Aktuell/GB/MH_13108_aktuell.pdf (visited on 07/30/2026) (cit. on p. 18).

[85] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903 (visited on 08/01/2026) (cit. on pp. 11, 34).

[86] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search*. 2025. arXiv: 2504.08066 [cs.AI]. URL: https://arxiv.org/abs/2504.08066 (cit. on p. 5).

[87] Xiang Yue, Yuansheng Ni, Kai Zhang, et al. *MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI*. 2024. arXiv: 2311.16502 [cs.CL]. URL: https://arxiv.org/abs/2311.16502 (visited on 07/29/2026) (cit. on p. 13).

[88] Wayne Xin Zhao, Kun Zhou, Junyi Li, et al. *A Survey of Large Language Models*. 2026. arXiv: 2303.18223 [cs.CL]. URL: https://arxiv.org/abs/2303.18223 (cit. on p. 7).

[89] Minhui Zhu, Minyang Tian, Xiaocheng Yang, et al. *Probing the Critical Point (CritPt) of AI Reasoning: a Frontier Physics Research Benchmark*. 2025. arXiv: 2509.26574 [cs.AI]. URL: https://arxiv.org/abs/2509.26574 (cit. on pp. 14, 21–23, 32–34).

A. Appendix

A.1. Outputs from the Example Run

A.1.1. Papers Downloaded

1. "Edelstein Effect in Isotropic and Anisotropic Rashba Models" [41]
2. "Out-of-plane Edelstein effects: Electric field induced magnetization in wave magnets" [39]
3. "Acoustic Rashba–Edelstein effect" [40]
4. "Spin accumulation at nonmagnetic interface induced by direct Rashba–Edelstein effect" [13]
5. "Electric-Dipole Spin Resonances" [71]
6. "Out-of-Plane Spin Polarization from In-Plane Electric and Magnetic Fields" [38]

The original output is available at [70].

A.1.2. Extracted Information

These additional sources are listed in the extracted information:

7. "Properties of a 2D electron gas with lifted spectral degeneracy" [17]
8. "Spin polarization of conduction electrons induced by electric current in two-dimensional asymmetric electron systems" [36]
9. "Current-induced spin-orbit torques" [42]

The Extracted Rashba Hamiltonian as Stated in the Output

The **Rashba Hamiltonian** for a 2DEG at the Γ point is [Bychkov & Rashba, *JETP Lett.* 39, 78 (1984)]:

$$\boxed{\hat{H} = \frac{p^2}{2m} + \alpha\, \hat{z} \cdot (\mathbf{p} \times \sigma)}$$

where:

- m is the effective carrier mass,
- α is the Rashba spin–orbit coupling (SOC) strength,
- $\mathbf{p} = \hbar\mathbf{k}$ is the momentum,
- $\sigma = (\sigma_x, \sigma_y, \sigma_z)$ are the Pauli matrices,
- $\hat{z}$ is the direction perpendicular to the 2D plane.

Most of the core findings are attributed to the work of Gaiardoni et al. [41], including the *energy dispersion*, the *spin eigenstates*, and the *Fermi wavenumbers* split into high density regime (HDR) and low density regime (LDR). The general formula for the magnetization, which is stated to be the total spin density instead of the **magnetization density**, is referenced to both [41] and [39]. The following analytical results fo the isotropic case are provided:

Analytical Magnetization as Stated in the Output

4.1 Magnetization in the High-Density Regime (HDR)

For an electric field $\mathbf{E} = E_x\hat{x}$, the spin density along $\hat{y}$ in the HDR (both bands occupied) is [Gaiardoni et al., arXiv:2503.20712 (2025)]:

$$\boxed{M_y = \frac{\mu_B|e|\tau}{2\pi}\, m\alpha\, [\hat{z}\times\mathbf{E}]_y}$$

Key result: In the HDR, the magnetization is **constant, independent of the Fermi energy** E_F, and proportional to the Rashba SOC parameter α and the effective mass m.

4.2 Magnetization in the Low-Density Regime (LDR)

In the LDR (only the lower band occupied) [Gaiardoni et al., arXiv:2503.20712 (2025)]:

$$\boxed{M_y = \frac{\mu_B|e|\tau}{2\pi}\, \sqrt{m^2\alpha^2 + 2mE_F}\, [\hat{z}\times\mathbf{E}]_y}$$

For small E_F near the band crossing, this can be expanded as:

$$M_y \approx \frac{\mu_B|e|\tau}{2\pi}\left(\alpha m + \frac{1}{2}\frac{E_F}{\alpha}\right)[\hat{z}\times\mathbf{E}]_y$$

The spin density **increases linearly with the Fermi energy**. In the limit $\alpha \to 0$, the second term vanishes because E_F has a quadratic dependence on α.

Further information include the *Anisotropic Rashba Model*, the *Out-of-Plane Edelstein Effect*, the *Acoustic Edelstein Effect*, and the *Effect of a Magnetic Field*. The parameter dependencies are discussed for both the magnetization M and the Edelstein susceptibility χ_{xy}. Lastly, the Task provides guidelines for a numerical implementation along with code snippets and graphics it suggests to be implemented. The full report is available on GitHub at [63]

A.1.3. Unit Check

Unit Check of the Rashba Hamiltonian as Stated in the Output

Formula

$$\hat{H} = \frac{p^2}{2m} + \alpha\,\hat{z}\cdot(\mathbf{p}\times\sigma)$$

Quantities and Units

- H (Hamiltonian): Energy $\rightarrow [E]$ (Joules)
- p (Momentum): $[p] = [M][L][T]^{-1}$
- m (Mass): $[m] = [M]$
- α (Rashba parameter): $[\alpha] = ?$
- σ (Pauli matrices): Dimensionless

Tool Input

equation: H = p**2 / (2*m) + alpha * (p)
dimensions: ”H”: ”energy”, ”p”: ”momentum”, ”m”: ”mass”, ”alpha”: ”alpha_dimension”
unitList: energy, momentum, mass, alpha_dimension

Tool Output

2*energy*mass/(momentum*(2*alpha_dimension*mass + momentum))

Analysis The first term $\frac{p^2}{2m}$ has units $\frac{([M][L][T]^{-1})^2}{[M]} = [M][L]^2[T]^{-2} = [E]$. The tool output shows that for consistency, *alpha_dimension* must satisfy *energy* = *momentum* * *alpha_dimension*. Since $[E] = [M][L]^2[T]^{-2}$ and $[p] = [M][L][T]^{-1}$, we must have:

$$[\alpha] = \frac{[E]}{[p]} = \frac{[M][L]^2[T]^{-2}}{[M][L][T]^{-1}} = [L][T]^{-1}$$

Conclusion: The Rashba parameter α has units of **velocity** (m/s)

The energy dispersion is deemed dimensionally consistent by the tool output and no correction is applied. For the analytical magnetization formula, the Agent describes its process as follows:

Unit Correction for the HDR Magnetization as Stated in the Output

Formula

$$M_y = \frac{\mu_B |e| \tau}{2\pi} m\alpha \, [\hat{z} \times \mathbf{E}]_y$$

For dimensional analysis, we examine the product $M_y \propto \mu_B e \tau m \alpha E$.

Quantities and Units

- M_y (Magnetization/Spin Density): Magnetic moment per unit volume.
- In SI (Volume): $[M] = [I][L]^2$ (Am^2) / $[L]^3 = [I][L]^{-1}$ (A/m).
- In 2D (Area): $[M] = [I]$ (Amperes) / $[L]^2 = [I][L]^{-2}$ (A/m^2).
- In microscopic units (just density): $[M] \propto [\hbar]^{-1}[L]^{-2}$.
- μ_B (Bohr magneton): $[\mu_B] = [I][L]^2$ (Am^2 or J/T).
- e (Elementary charge): $[e] = [I][T]$.
- τ (Relaxation time): $[\tau] = [T]$.
- m (Mass): $[m] = [M]$.
- α (Velocity): $[\alpha] = [L][T]^{-1}$.
- E (Electric field): $[E] = [M][L]^3[T]^{-3}[I]^{-1}$ (V/m).

Tool Input

equation: M_y = mu_B * e * tau * m * alpha * E
dimensions: ”M_y”: ”magnetization”, ”mu_B”: ”magnetic_moment”, ”e”: ”charge”, ”tau”: ”time”, ”m”: ”mass”, ”alpha”: ”velocity”, ”E”: ”electric_field”
unitList: magnetization, magnetic_moment, charge, time, mass, velocity, electric_field

Tool Output

magnetization*exp(-1)/(charge*magnetic_moment*mass*time*velocity*electric_field)

...omitting a long analysis which is part of the original output...

Conclusion on Formula Consistency: The formula is dimensionally consistent **if and only if** the system of units uses $\hbar = 1$ (natural units) or the parameters m and α are defined in units that include $\hbar$ (common in computational materials science, e.g., α in $eV \cdot \AA$ and m in $eV^{-1}\AA^{-2} \cdot \hbar^2$). However, in standard SI units, the formula is missing a factor of $\hbar^{-2}$.
Correction for Standard Units:

$$M_y = \frac{\mu_B |e| \tau}{2\pi\hbar^2} m\alpha \left[\hat{z} \times \mathbf{E}\right]_y$$

The same $\frac{1}{\hbar^2}$ correction is applied to the expression for the LDR magnetization. Additionally, formulas for the anisotropic susceptibility and the Out-of-Plane susceptibility are analysed, with the output not stating that tool calls were made during the process and failing to provide a unit system for them. Lastly, the findings are summarised, restating the $\frac{1}{\hbar^2}$ correction for both HDR and LDR magnetization. The full report can be inspected at [65].

A.1.4. Verification of the Unit Check for the Magnetization Magnitude

This unit check correctly identifies the Rashba coupling strength α to be a velocity for the given Hamiltonian (see subsubsection A.1.3). The unit correction applied to the magnetization formulas achieves unit consistency, although some steps along the way are worth being discussed. The assumed units for the magnetization are not correct: The extracted information (see subsubsection A.1.2) state that M_y is the total spin density, which is also incorrect, since the magnetization is the magnetic moment (Am^2) per unit volume. The formula is referenced to Gaiardoni et al. [41], who discuss a 2D system, which has a unit volume with dimensions of Length^2. Therefore, the unit for the magnetization is $\frac{\mathrm{Am}^2}{\mathrm{m}^2} = \mathrm{A}$.

In the following, the correction applied to the magnetization by the Physics Modelling Assistant is verified by hand. The used properties are the following:

- M_y is the magnetization (Am^2) per 2D unit volume, resulting in its unit being A.
- μ_B is the Bohr magneton. It is expressed in JT^{-1}.
- e is the charge with the unit of A s.
- τ is the transport time expressed in s.
- m is the effective mass in kg.
- α is the Rashba coupling strength. As stated in subsubsection A.1.3, it has units of velocity being ms^{-1}.
- $\hat{z}$ is a dimensionless direction.
- $\mathbf{E}$ is the applied electric field. It can be expressed in Vm^{-1}.
- $\hbar$ is the Planck constant with SI units of J s.

The relevant unit conversions are

$$1\mathrm{T} = 1\frac{\mathrm{V\,s}}{\mathrm{m}^2} \text{ and} \tag{A.1.1}$$

$$1\mathrm{J} = 1\mathrm{kg}\frac{\mathrm{m}^2}{\mathrm{s}^2}. \tag{A.1.2}$$

This allows for the following derivation steps:

$$
\begin{aligned}
[M_y] &\overset{!}{\hat{=}} \left[\mu_B |e| \tau m \alpha \left(\hat{z} \times \mathbf{E}\right)_y \frac{1}{2\pi\hbar^2}\right] \\
\mathrm{A} &\overset{!}{\hat{=}} \frac{\mathrm{J}}{\mathrm{T}} \cdot \mathrm{As} \cdot \mathrm{s} \cdot \mathrm{kg} \cdot \frac{\mathrm{m}}{\mathrm{s}} \cdot \frac{\mathrm{V}}{\mathrm{m}} \cdot \frac{1}{\mathrm{J}^2\mathrm{s}^2} \\
&\text{now use Equation A.1.1:} \\
&\overset{!}{\hat{=}} \frac{\mathrm{Jm}^2}{\mathrm{Vs}} \cdot \mathrm{As} \cdot \mathrm{s} \cdot \mathrm{kg} \cdot \frac{\mathrm{m}}{\mathrm{s}} \cdot \frac{\mathrm{V}}{\mathrm{m}} \cdot \frac{1}{\mathrm{J}^2\mathrm{s}^2} \\
&\overset{!}{\hat{=}} \frac{\mathrm{Jm}^2}{\mathrm{Vs}} \cdot \mathrm{As} \cdot \mathrm{s} \cdot \mathrm{kg} \cdot \frac{\mathrm{m}}{\mathrm{s}} \cdot \frac{\mathrm{V}}{\mathrm{m}} \cdot \frac{1}{\mathrm{J}^2\mathrm{s}^2} \\
&\overset{!}{\hat{=}} \mathrm{A} \frac{\mathrm{kgm}^3\mathrm{s}^2}{\mathrm{Js}^4\mathrm{m}} \\
&\overset{!}{\hat{=}} \mathrm{A} \frac{1}{\mathrm{J}} \cdot \mathrm{kg} \frac{\mathrm{m}^2}{\mathrm{s}^2} \\
&\text{now use Equation A.1.2:} \\
&\overset{!}{\hat{=}} \mathrm{A} \quad \text{verifies unit consistency}
\end{aligned} \tag{A.1.3}
$$

If $[\alpha] = \mathrm{eV\mathring{A}} \overset{!}{\hat{=}} \mathrm{J\,m}$ as stated in “Edelstein Effect in Isotropic and Anisotropic Rashba Models” [41], an additional $\frac{1}{\hbar}$ correction is required, resulting in $\left[\frac{\alpha}{\hbar}\right] \overset{!}{\hat{=}} \frac{\mathrm{Jm}}{\mathrm{Js}} \overset{!}{\hat{=}} \frac{\mathrm{m}}{\mathrm{s}}$.

A.1.5. Suggested Parameters

For each parameter the Task lists its range, a specific suggestion, and a justification for its choice. Some of the justifications include sources backing them, this is the case for the effective mass m and the Rashba coupling strength α. There are two different unit suggestions for the effective mass, with the first one being $0.13 \frac{1}{\mathrm{eV\mathring{A}}^2}$, while the second is $0\,5 \cdot 9\,11 \cdot 10^{-31}\mathrm{kg}$. The final parameter suggestions are the following:

Parameter Suggestions for the Implementation as Stated in the Output

Parameter	Symbol	Value	Units	Physical Source
Effective Mass	m	$0.5\,m_e$	kg	$m_e = 9.11 \times 10^{-31}$ kg. Effective masses in LAO/STO range from 0.5 to $3m_e$ [1].
Rashba Coupling	α	5.0×10^{-11}	eV·m	Corresponds to ≈ 50 meV·$\mathring{A}$. This is a ”strong” SOC regime observed in Bi/Ag or gated interfaces [2, 3].
Fermi Energy	E_F	60	meV	Typical carrier density $n \sim 10^{13}$ cm^{-2}. Places system in HDR ($E_F \gg E_R$).
Relaxation Time	τ	1.0×10^{-12}	s	1 picosecond. Corresponds to a mobility $\mu \approx 10$ cm^2/V·s for $m \approx m_e$ [1].
Electric Field	E_x	1000	V/m	Linear response regime. Small enough to avoid Joule heating, large enough for signal.
g-factor	g	2.0	-	Free electron value.
Temperature	T	4.2	K	Low temperature limit (often used in theoretical/comparison papers).

Sources:

1. **Caprara, S. et al.** *Rep. Prog. Phys.* **79**, 096501 (2016). (Review of LAO/STO parameters: masses, mobilities, densities).

2. **Ast, C. R. et al.** *Phys. Rev. Lett.* **98**, 186807 (2007). (Giant Rashba splitting on Bi/Ag surface, $\alpha \sim 3$ eV$\cdot\mathring{A}$).

3. **Ben Shalom, M. et al.** *Phys. Rev. Lett.* **105**, 206801 (2010). (Tunable α and E_F in LAO/STO).

The full report is accessible via GitHub [66].

With the exception of the Rashba coupling strength and the effective mass, the suggested parameters are in line with the previous findings. The initial suggestion for the effective mass m to be expressed in $\frac{1}{\text{eV}\mathring{\text{A}}^2}$ is justified when working in atomic units. Contradicting the behaviour of the Task's definition, the suggested units for the Rasba coupling strength α of $\frac{\text{m}}{\text{s}}$ are not adopted, instead the suggested parameter uses eV m. As discussed in subsubsection A.1.4, this choice requires an additional correction of $\frac{1}{\hbar}$ to the formula for the magnetization (see subsubsection A.1.3). These parameter choices appear to have been influenced by the additional sources stated, but the Task was not able to adapt them to the unit system proposed by the unit check (see subsubsection A.1.3).

The offered *physical sources* contain a two wrong calculations. The first one relates to the Rashba coupling strength α; the suggested value of $5.0 \cdot 10^{-11}$eV $\cdot$ m is wrongly converted to $50\text{meV} \cdot \text{Å}$, while the correct conversion results in $\alpha = 500\text{meV} \cdot \text{Å}$. The second error is the claimed mobility which is said to be calculated based on the relaxation time. Using the Drude expression [74]

$$\mu = \frac{e\tau}{m_e} \tag{A.1.4}$$

where μ is the mobility, e is the carrier charge, τ is the transport time, and m_e is the effective mass to calculate μ according to the stated specifications $m_e = 9.11 \cdot 10^{-31}$kg, $\tau = 10^{-12}$s, and the previously set $e = 1.60 \cdot 10^{-19}$C results in $\mu = 0.1756 \frac{\text{m}^2}{\text{Vs}} = 1756 \frac{(\text{cm})^2}{\text{Vs}}$, which widely differs from the Task's claim of $\mu \approx 10 \frac{(\text{cm})^2}{\text{Vs}}$

The first and the third of the stated sources could not be found, they are likely a mix-up of multiple others. The second on is valid and can be found under [12]. For the third source, at least two very similar ones exist: "Tuning Spin-Orbit Coupling and Superconductivity at the $SrTiO_3/LaAlO_3$ Interface: A Magnetotransport Study" by Ben Shalom et al. [16] is closest in terms of the content, but the identifier 104.126802 does not match. "Shubnikov–De Haas Oscillations in $SrTiO_3/LaAlO_3$ Interface" by Ben Shalom et al. [15] does not discuss the claimed content, but its bibliographic information match except for the identifier which is 105.206401. The paper associated with the identifier 206801 is "Theory of High-Energy Features in the Tunneling Spectra of Quantum-Hall Systems" by MacDonald [60].

A.1.6. Implementation and Simulation

The two simulation Tasks create two functionally identical files with only minor differences regarding the comments. On execution the code calculates the Rashba effect induced change E_R of the minimum energy. It is compared to the Fermi energy to determine if the system is in the high density regime (HDR) or low density regime (LDR), with the Fermi energy being higher than E_R indicating the HDR. The code's output is a graphic showing various dependencies for the Rashba effect. To access the unmodified code, see [67]. It is executable but requires some corrections. The code uses the Rashba coupling strength in J m,

but does not apply the additional correction of $\frac{1}{\hbar}$ (see subsubsection A.1.4). Therefore, the initial results were off by this factor, with a magnetization M_y in the range of 10^{-44} $\frac{\mathrm{A}}{\mathrm{m}}$. The unit for the magnetization was also incorrect; as stated in subsubsection A.1.3, it should be A. To obtain more meaningful results, the following modifications were applied by hand:

- the magnetization formulas for both HDR and LDR got an additional $\frac{1}{\hbar}$ factor,
- the axis labels for the unit of the magnetization M_y was changed to Am^2/m^2,
- the axis labels for the unit of the Rashba coupling strength α were fixed from *eV·\AA* to *eV·A*.

The resulting graphic is shown in Figure 3. The corrected code is accessible via [69].

Declaration of authorship

I hereby declare that I have written this thesis independently and without unauthorized assistance.

All sources and references used have been properly cited.

This thesis has not been submitted in the same or substantially similar form for any other degree or academic qualification.

Halle (Saale), 22nd August 2026

_____

Janis Felix Jacobi

Acknowledgements

The author gratefully acknowledge the computing time granted by the KISSKI project. The calculations for this research were conducted with computing resources under the project 9e28b773-a710-4a17-a242-523e92c812de.

I want to express my gratitude to my supervisor, Professor Schröter, for the opportunity to write my Bachelor's Thesis under his supervision and his valuable suggestions regarding the direction of my work.

My sincere thanks go to my second supervisor, Professor Lounis, for his valuable feedback regarding my Bachelor's Thesis, especially for his suggestions on how to design the unit check.

I greatly appreciate the feedback received from Moritz Winterott regarding the design of the Physis Modelling Agent as well as his proofreading of my Thesis.

Lastly, want to thank my parents for supporting me throughout my studies, and my girlfriend for proof-reading my Thesis as well as taking the time to listen to my confusions and successes encountered.